



Solution Design & Delivery Package

PRD · SDD · Implementation Plan · Tasks · Schema · Test Plan · Migration
Runbook · ADRs

FileMaker Advantage → psg-hub Integration

This spec package defines how PSG's legacy FileMaker Advantage Program (the Advantage, Survey, Import Flush, and Web files) is re-platformed into psg-hub as native ops modules, plus the optional historical data migration. It follows the spec-driven workflow: SPECIFY → PLAN → TASKS → IMPLEMENT.

The behavioral source of truth is the DDR analysis in `agents/CI0/FileMaker-Analysis-2026-06-08/`. This package turns that analysis into a build the team can execute.

Read in this order

#	Document	Purpose
1	<code>spec.md</code>	PRD: objective, users, scope by milestone, 31 functional requirements, success criteria, boundaries, open questions
2	<code>solution-design.md</code>	SDD: architecture, schema deltas, interfaces, runtime flows, ADR summary, acceptance criteria
3	<code>plan.md</code>	Implementation plan: components, build order, milestone mapping, parallel vs sequential, risks
4	<code>tasks.md</code>	Discrete tasks per milestone with acceptance + verify + files
5	<code>schema/migrations.md</code>	Forward-only Postgres DDL for the table deltas
6	<code>test-plan.md</code>	Test strategy, EARS acceptance criteria, concrete unit cases, migration reconciliation
7	<code>data-migration-runbook.md</code>	Optional v1.3.5 historical migration runbook
8	<code>adr/ADR-001..005.md</code>	Standalone architecture decision records (all Proposed)

What this replaces

FileMaker subsystem	psg-hub target	Milestone
Advantage letters + print/production	Production module + Lob/SendGrid + <code>production_batches</code>	v1.3
Survey scoring + Perfect/Misfire/HotSpot/Unresolved/Referral alerts	Survey module + scoring service + <code>survey_responses</code>	v1.1 / v1.4
Time-based eligibility + survey suppression	<code>letter_eligibility</code> + eligibility cron	v1.1 / v1.3
26 operational reports	Parameter-driven report engine	v1.4
Import Flush (325 per-shop importers)	<code>import_templates</code> + psg-import	v1.1
Web PSOS scheduler	Vercel cron	v1.1+
Historical data (281K customers, 334K surveys)	Supabase via psg-data-lake worker	v1.3.5 (optional)

Status

Draft. The five ADRs are Proposed and need Nick's sign-off before implementation. Open questions are listed in `spec.md` (OQ-1..OQ-5): migration scope, source of `referral_tracking_enabled` / `credit_hold`, which alert-script generation is authoritative, the Lob vs in-house mail split, and the end-consumer PII retention policy.

Key decisions (see adr/ for full records)

1. Re-platform behavior, do not port FileMaker code or schema 1:1.
2. Replace the composite string match key with a real foreign key.
3. Move automation to Vercel cron + idempotent Postgres workers.
4. Make the survey-alert suppression rule explicit and auditable.
5. Treat historical migration as an optional v1.3.5 add-on.

Provenance

Derived from the DDR dated 2025-05-27 and the analysis pack at `agents/CI0/FileMaker-Analysis-2026-06-08/`. Aligned to `PLANNING.md` (71 design decisions, ops milestones v1.1–v1.4, Q15).

Product Requirements Document

This PRD specifies what psg-hub must build to replace the legacy FileMaker Advantage Program with native ops modules. It pairs with the Solution Design Document (`solution-design.md`) as the source of truth for architecture, and with the DDR analysis pack (`agents/CI0/FileMaker-Analysis-2026-06-08/`) as the source of behavior. Where this PRD and the SDD use a term, they use it identically.

Objective

PSG runs its core customer-experience program on a four-file FileMaker system: time-based customer letters, post-repair surveys, an alert subsystem that classifies each survey, and a feedback loop where an open survey alert removes a customer from routine mail. This system is the daily driver for PSG account managers, the production team, and management reporting. It carries 281K repair customers and 334K surveys, runs on a single server-side scheduler, and depends on fragile composite match keys and date-stamp idempotency.

We are rebuilding that system natively inside psg-hub as ops modules over the shared Supabase schema. We are re-platforming behavior, not porting code. Each FileMaker subsystem maps to an already-planned psg-hub module (Ops backbone, Surveys, Production, Reports, Imports) and table set. The DDR analysis supplies the exact rules those modules must encode: letter eligibility windows, the survey alert classifications, the suppression rule, and the printed/posted idempotency stamps.

Success looks like this. PSG account managers, the production team, and billing ops run their full daily workflow in psg-hub. The survey-to-letter feedback loop runs as Vercel cron plus idempotent Postgres workers, with no double-sends and no silently broken customer-to-survey links. All 26 named reports render from one parameter-driven engine. FileMaker is retired as the daily driver at the v1.3 cutover and stays read-only for historical access. Historical data migration is optional add-on scope (v1.3.5) and does not block functional replacement.

Users and Personas

The daily drivers are internal PSG staff. End consumers receive mail and surveys but never log in.

Persona	Role	What they do in psg-hub	Primary milestones
Account manager	PSG internal	Manage companies, employees, programs, repair customers, ROs, estimates; enter and review surveys; resolve the needs-match queue	v1.1, v1.4
Production team	PSG internal	Build production batches, print and mail letters via Lob or in-house, manage reprints, search production history	v1.3
Billing ops	PSG internal	Run invoicing and volume reports; reconcile production and pay-type data	v1.4
Strategy	PSG internal	Run CSI, survey-alert, and referral reports; act on Perfect Score, Mis-Fire, Hot Spot, Unresolved Issue, and Referral Noted outcomes	v1.4
Superadmin (Nick, Tina, Brian)	PSG internal	Manage roles, shop assignments, module access, and security profiles that gate ops functions	v1.5
End consumer (repair customer)	External	Receives PSG-printed mail and paper or web surveys; does NOT log in; tracked as a <code>repair_customers</code> entity with no UI surface	tracked across all ops milestones

Access for internal personas is gated by Supabase Auth plus `security_profiles.functions_jsonb` flags (`manage_companies`, `manage_surveys`, `manage_production`, `manage_reports`, `manage_data_imports`, and related flags). RLS clamps customer data to authorized shops.

Background

The legacy system is four FileMaker files, hub-and-spoke around Advantage. The DDR export dated 5/27/2025 confirms the scale and the wiring (see `03 - Cross-File Architecture Analysis.md`).

File	Scripts	Core table	Records (approx)	Role
Advantage_06.1	1,396	Repair Customer	281,077	Letter engine, data hub
Survey_06.1	921	Survey Input	334,269	Survey scoring, alerts, reporting
Import Flush	452	(staging)	~12	Per-shop ETL conduit into Advantage
Web	22	(web aggregate)	~107,000	Server-side trigger and scheduler

Advantage runs time-based customer letters: 3 month, 6 month, 1 year, 18 month, 2 year, birthday, drivers license, thank-you, and referral. Survey scores each post-repair survey and raises one of five alert classes: Perfect, Misfire, Hot Spot, Unresolved, Referral. The two files join on a composite match key, shop or master id plus creation month plus creation year, reconstructed in the DDR from the match-key calcs (`RC_MatchField_survey` on the Advantage side, `S_RC_MatchField_RepairCust` on the Survey side).

The survey-to-letter feedback loop is the load-bearing behavior. A completed survey is scored and classified; the classification reads Advantage `Master Client` flags across the file for the referral and credit-hold gates. The nightly Advantage `SS - Eligibility - *` scripts read the survey alert state live over the relationship and set routine letter eligibility to 0 when an open alert exists. So an open survey alert silently removes a customer from routine mail, with no human in the loop (see `03 - Cross-File Architecture Analysis.md` section 3). Alert and referral letters are produced as batch print runs that stamp post-date fields (`SQ_Alert_PostDate_*`, `S_RC_Referral_Letter_Post`) so the same record is not re-sent.

There is no in-file script-trigger layer to preserve. The DDR confirms 0 layout triggers, 12 object `onObjectExit` field validations, and 1 in-file PSOS. Automation comes three ways: the Web file firing Perform Script on Server into Advantage eligibility scripts on a FileMaker Server schedule, date-stamp incremental processing, and live cross-file relationship reads (see `03 - Cross-File Architecture Analysis.md` section 4). psg-hub must rebuild this automation as Vercel cron plus Postgres, not port triggers.

The DDR also flagged debt we must not reproduce: 325 per-shop import scripts, triplicate per-letter scripts (Preview/Print/Loop), three live generations of survey alert scripts (v2, WIP, v3 LIVE), a composite string match key that breaks links silently, and stale external references (BridgeSystem, EmployeeSatisfaction, Import Flush2, dormant `PSG_SurveyExport_07.1`, a broken "Phoenix Solutions Group" reference).

In-Scope and Out-of-Scope

Scope is organized by milestone. The customer track (v0.1 to v0.4) ships first and is out of scope for this PRD. The ops backbone that replaces FileMaker ships v1.1 to v1.4, with optional historical migration at v1.3.5.

v1.1 Ops Foundation

In scope: companies, employees, company programs, products, items, repair customers, repair orders, estimates, surveys (entry and view), system-configuration master data (vehicles, insurance companies,

insurance agents), RO and estimate import driven by `import_templates` (absorbs psg-import), and security profiles. Parallel run with FileMaker; FileMaker stays authoritative.

Out of scope for v1.1: production batches, mail and email sending, the 26 reports, historical data migration.

v1.3 Production and FileMaker Cutover

In scope: production batches and documents, the dual mail adapter (Lob.com plus in-house print queue), mail-merge templates, the print queue with idempotent status and reprint audit, historical production search, the survey-ingest and eligibility-compute cron, the suppression rule made explicit, and the v1.3 cutover that points the production team at psg-hub for all new production. FileMaker becomes read-only for history.

Out of scope for v1.3: the 26 reports (v1.4), historical data migration (v1.3.5).

v1.4 Operational Reports

In scope: the parameter-driven report engine and all 26 named reports, including Perfect Score, Mis-Fire, Hot Spot, Unresolved Issue, Referral Noted, and Referral Comparison, with date-range and filter UI and CSV/Excel/PDF export.

Out of scope for v1.4: new analytical reports beyond the 26; write-back of report results into ops tables.

v1.3.5 Historical Migration (optional add-on)

In scope if triggered: ETL through `psg-data-lake` that extracts from the DDR CSV exports plus the FileMaker Data API, resolves legacy match keys to foreign keys, redacts PII, dedupes, loads historical entities, and reconciles counts to the DDR (`repair_customers` ~281K, `survey_responses` ~334K) before cutover relies on it.

Out of scope for v1.3.5: any change to functional replacement; v1.1 to v1.4 do not depend on this add-on.

Out of scope for the whole program

- Two-way sync back into FileMaker. psg-hub is downstream only during parallel run; it writes its own data and never writes back to FileMaker.
- End-consumer login. Repair customers receive mail and surveys; they have no UI surface and no account.
- Porting FileMaker scripts or schema 1:1. The four-file split collapses into logical modules on one Supabase schema.
- Recreating stale FileMaker external references (BridgeSystem, EmployeeSatisfaction, Import Flush2, `PSG_SurveyExport_07.1`, "Phoenix Solutions Group").

Functional Requirements

Requirements are grouped by subsystem. Each is concrete and testable. Table and column names match the SDD and PLANNING.md.

Ops backbone

FR-1. The system shall provide CRUD for `companies`, `employees`, and `company_programs`, where a company is the primary internal ops entity with a system-generated unique Shop ID, and a company program enrolls a company in a product with `customizations_jsonb` overrides (logo, header, footer, greeting).

FR-2. The system shall provide CRUD for `repair_customers`, where each record holds end-consumer name, address, phone, and email, and exposes no end-consumer login surface.

FR-3. The system shall provide CRUD plus list, filter, sort, and search for `repair_orders`, including add-new, preview, cancel, and add-additional-document actions, with each RO linked to a repair customer, company, vehicle, insurance company, and insurance agent.

FR-4. The system shall provide CRUD plus list and detail for `estimates`, each linked to a repair customer and company.

FR-5. The system shall provide CRUD for system-configuration master data: `products`, `items`, `vehicles`, `insurance_companies`, and `insurance_agents`.

FR-6. The system shall keep `repair_orders` completion date and `survey_responses` submitted date as distinct date axes, because eligibility windows use RO completion date and never survey date.

Surveys and alert classification

FR-7. The system shall store surveys in `survey_responses` extended with `repair_customer_id` (a real foreign key replacing the composite string match key), `alert_class`, `csi_resolve`, `would_recommend`, `unresolved_shop`, `referral_consumer`, `alert_posted_at`, and `referral_letter_posted_at`.

FR-8. The system shall resolve each ingested survey to a `repair_customer_id` by foreign key, falling back to `legacy_match_key` during migration, and shall set `alert_class` using the DDR classification rules.

FR-9. The system shall classify each survey into exactly one of `perfect`, `misfire`, `hotspot`, `unresolved`, `referral`, or `none`, applying the DDR thresholds: a satisfied, would-recommend customer with `csi_resolve = 1` is Perfect; satisfied and would-recommend but `csi_resolve < 1` is Misfire (a missed referral); `unresolved_shop` true is Unresolved; a negative result (`csi_resolve < 1`) is Hot Spot; and a happy, would-refer customer is Referral only where the company has `referral_tracking_enabled` and the customer is not on `credit_hold`.

FR-10. The system shall stamp `alert_posted_at` when a survey alert is posted, so the same alert is not re-raised, replacing the FileMaker `SQ_Alert_PostDate_*` date stamps.

FR-11. The system shall retain any survey that cannot be matched to a repair customer, leave its `repair_customer_id` null, and surface it in a needs-match ops queue; the system shall never silently drop an unmatched survey.

FR-12. The system shall seed `referral_tracking_enabled` and `credit_hold` onto `company_programs` and `repair_customers` before classification runs, because the Referral versus Misfire branch reads both and misclassifies every survey if either is missing.

Letter eligibility and suppression

FR-13. The system shall compute `letter_eligibility` rows for each `letter_kind` (`three_month`, `six_month`, `one_year`, `eighteen_month`, `two_year`, `birthday`, `drivers_license`, `thank_you`, `referral`) by finding repair customers whose RO completion date falls in the kind's date window.

FR-14. The system shall suppress a customer from a routine letter when an open survey alert exists in the window, set `eligible = false` and `suppressed_by_alert = true`, and record the suppression explicitly rather than as an emergent join side effect.

FR-15. The system shall enforce a unique constraint on (`repair_customer_id`, `letter_kind`, `period_key`) and upsert eligibility idempotently, so re-running the compute never creates a duplicate letter for the same customer, letter kind, and period.

FR-16. The system shall stamp `printed_at` on a `letter_eligibility` row only when its letter is produced, replacing the FileMaker `Letter_*_Printed` date stamps, and shall skip rows already stamped.

Production and mail/email

FR-17. The system shall build a `production_batch` from a set of products and an optional set of companies (blank means all companies), create one `production_document` per generated letter with a unique `print_id`, and return the batch id and document count.

FR-18. The system shall transition only `unprinted` documents to `printed`, stamp `printed_at` and `printed_by_profile_id`, and report counts of printed and skipped-already-printed, so a re-triggered print is idempotent.

FR-19. The system shall send physical mail through a dual `MailAdapter` (Lob.com or in-house print queue), select the vendor per template or per shop, and record each send in `mail_vendor_jobs` with vendor, external job id, and status.

FR-20. The system shall send letter and report email through SendGrid and record each send in `email_jobs`; the system shall send SMS reminders and production-status messages through Twilio and record each in `sms_jobs`.

FR-21. The system shall accept Lob delivery-status webhooks at `/api/webhooks/lob`, verify the HMAC signature, and update the matching `mail_vendor_jobs` row; on a failed vendor send the system shall mark the job failed with the error, leave the document `unprinted`, and retry with backoff.

FR-22. The system shall log every reprint in `production_reprint_log` with the reprinting profile and timestamp.

Reports engine and the 26 reports

FR-23. The system shall serve reports from one parameter-driven engine at `/api/reports/[reportSlug]` (and `/ops/reports/[reportSlug]`), parameterized by date range and filters, replacing the FileMaker `PDF Generate Report(ReportName)` and `Email Report` scripts.

FR-24. The system shall serve all 26 named reports: `processing-recap`, `invoicing-recap`, `reprint-recap`, `pay-type-analysis`, `vehicle-analysis-make`, `vehicle-analysis-model`, `referral-directory`, `recap-trailing`, `agent-capture`, `agent-sales`, `claims-review`, `audit`, `name-recap-by-shop`, `referral-comparison`, `monthly-csi-display`, `performance-dashboard`, `market-dashboard`, `estimator-csi`, `body-tech-performance`, `painter-performance`, `survey-alert-recap`, `rental-car-analysis`, `perfect-score`, `mis-fire`, `hot-spot`, `unresolved-issue`, and `referral-noted`.

FR-25. The system shall derive the five survey-alert reports (Perfect Score, Mis-Fire, Hot Spot, Unresolved Issue, Referral Noted) and Referral Comparison from `survey_responses.alert_class`, so report output matches the classification used by the feedback loop.

FR-26. The system shall export every report to CSV, Excel, and PDF.

Imports

FR-27. The system shall drive RO and estimate import from `import_templates`, one per company and kind (`ro` or `estimate`), each holding a `field_mapping_jsonb`, replacing the 325 per-shop FileMaker import scripts.

FR-28. The system shall accept import file uploads (xlsb, xlsx, csv, txt), validate each row against the company's template before load, and absorb the address-validation and smart-resolution logic from psg-import.

FR-29. The system shall source migration imports from the DDR exports and the FileMaker Data API only, never from a developer workstation or a machine-specific local path.

Automation (cron)

FR-30. The system shall run survey ingest, eligibility compute, production-batch status, and mail-vendor poll as Vercel cron jobs, each idempotent and window-based, replacing the Web file PSOS scheduler.

FR-31. The system shall make each cron self-healing: a missed run recovers on the next run because writes are idempotent upserts on unique constraints, and the system shall raise a freshness alert when `computed_at` is stale beyond 36 hours.

Non-Functional Requirements

Performance

NFR-1. LCP shall be under 3 seconds on `/ops/*` and `/ops/reports/*`.

NFR-2. The survey-ingest cron and the eligibility-compute cron shall each complete in under 5 minutes over the full population.

NFR-3. Report queries shall use the `pg` pool plus a Redis cache, not PostgREST, for heavy reads.

Reliability

NFR-4. Every cron and every webhook handler shall be idempotent, and the system shall guarantee zero double-sends, verified by unique constraints on `letter_eligibility` and by `printed_at` guards on `production_documents`.

NFR-5. The unmatched-survey rate shall be visible in the needs-match queue and shall be under 2 percent after migration reconciliation.

NFR-6. Every external call (Lob, SendGrid, Twilio, FileMaker Data API) shall use retry plus a circuit breaker, with no bare catch that swallows errors.

Security and PII

NFR-7. Ops tables shall be RLS-gated by `roles` plus `security_profiles.functions_jsonb` for `psg_internal` users, with `psg_superuser` bypass; customer data shall be RLS-clamped to authorized shops.

NFR-8. End-consumer PII in `repair_customers` and survey `raw_payload` shall follow the existing `psg_sensitive_pii_*` redaction patterns; OAuth and vendor secrets shall be encrypted at rest with pgsodium.

NFR-9. PII shall pass a manual sign-off at v1.1, v1.3, and v2.0.

Auditability

NFR-10. The system shall maintain an append-only `access_audit` trail and shall log production prints, reprints, and cancels separately.

NFR-11. The system shall record outbound communication in `email_jobs`, `sms_jobs`, and `mail_vendor_jobs`, and shall record Python worker runs in `python_worker_jobs`, giving the audit trail FileMaker lacked.

Success Criteria

Phrased as testable conditions; EARS phrasing where it fits.

SC-1. WHEN a survey is ingested, THE SYSTEM SHALL resolve its `repair_customer_id` and set `alert_class` using the DDR classification rules.

SC-2. IF a survey cannot be matched to a repair customer, THEN THE SYSTEM SHALL retain it and surface it in the needs-match queue, and the unmatched rate SHALL stay under 2 percent after reconciliation.

SC-3. WHEN eligibility is computed, THE SYSTEM SHALL exclude any customer with an open survey alert in the window and record `suppressed_by_alert = true`.

SC-4. THE SYSTEM SHALL never create a duplicate letter for the same `repair_customer_id`, `letter_kind`, and `period_key`.

SC-5. WHEN a batch is printed, THE SYSTEM SHALL transition only `unprinted` documents to `printed` and stamp `printed_at`.

SC-6. WHEN Lob returns a status, THE SYSTEM SHALL update the matching `mail_vendor_jobs` row within one webhook cycle.

SC-7. THE SYSTEM SHALL serve all 26 reports from the parameter-driven engine, and each report SHALL export cleanly to CSV, Excel, and PDF over a 12-month range.

SC-8. THE SYSTEM SHALL keep LCP under 3 seconds on `/ops/*` and `/ops/reports/*`.

SC-9. WHILE the parallel run is active, THE SYSTEM SHALL perform no write back to FileMaker.

SC-10. WHERE the v1.3.5 migration add-on is enabled, THE SYSTEM SHALL load historical entities and reconcile counts to the DDR (`repair_customers` ~281K, `survey_responses` ~334K) before cutover relies on the migrated data, and a spot check of 50 customer-to-survey links SHALL resolve correctly.

Boundaries

Always

- Always write idempotently. Every cron and webhook uses upserts on unique constraints or status-column guards.
- Always retain unmatched records and queue them; never silently drop a survey or an RO.
- Always keep RO completion date and survey date as distinct axes when computing eligibility.
- Always gate ops functions through `security_profiles.functions_jsonb` and clamp customer data with RLS.
- Always source migration data from the DDR exports and the FileMaker Data API.

Ask first

- Ask first before any schema change beyond the columns this PRD and the SDD already specify.
- Ask first before changing a survey alert threshold or an eligibility window, since these drive revenue and customer contact.
- Ask first before triggering the v1.3.5 historical migration add-on.
- Ask first before selecting or switching the mail vendor split (Lob versus in-house) for a template or shop.

Never

- Never write back to FileMaker during the parallel run; psg-hub is downstream only until the v1.3 cutover.
- Never expose an end-consumer login or UI surface for repair customers.
- Never recreate the retired FileMaker debt: the composite string match key, the 325 per-shop import scripts, the triplicate per-letter scripts, or the stale external references.
- Never reproduce the FileMaker date-stamp re-fire risk; a cleared or back-dated stamp must not re-send or skip a letter.

Open Questions

These remain unresolved and should be answered before the affected milestone locks.

OQ-1 (Q15, migration scope). Is the v1.3.5 historical migration full history or a cutoff date, and what are the audit-retention requirements? Decide only if the optional add-on is triggered.

OQ-2 (source of `referral_tracking_enabled` and `credit_hold`). In FileMaker these were read cross-file from `Master Client (M_ReferralNoted_flag, M_CreditHold_flag)`. Confirm the authoritative source per company and per customer, and confirm where each lands in psg-hub (`company_programs.referral_tracking_enabled` and `repair_customers.credit_hold` per the SDD) before classification runs.

OQ-3 (authoritative alert-script generation). The DDR found three live generations of survey alert scripts (v2, WIP, v3 LIVE). Confirm the v3 LIVE logic is the authoritative source before encoding `classifySurvey`, so the rebuilt classification matches what FileMaker actually ran.

OQ-4 (mail vendor split). Confirm the rule that selects Lob versus the in-house print queue per template or per shop, including any address-verification or cost thresholds that should drive the choice.

OQ-5 (end-consumer PII retention). Confirm the `repair_customers` PII retention policy: hard delete versus anonymize after N days. Tracked as a PLANNING operational open question and needed for the v1.3 PII sign-off.

Solution Design Document

Integrate the legacy FileMaker Advantage Program (the Advantage, Survey, Import Flush, and Web files) into psg-hub as native ops modules, and migrate its historical data. This SDD turns the DDR analysis in `agents/CI0/FileMaker-Analysis-2026-06-08/` into a build spec aligned to the psg-hub milestone plan (`projects/psg-hub/PLANNING.md`).

Validation Checklist

CRITICAL GATES (Must Pass)

- [x] All required sections are complete
- [x] No [NEEDS CLARIFICATION] markers remain
- [x] Architecture pattern is clearly stated with rationale
- [] **All architecture decisions confirmed by user** (ADRs below are Proposed — pending Nick's confirmation)
- [x] Every interface has specification

QUALITY CHECKS (Should Pass)

- [x] All context sources are listed with relevance ratings
 - [x] Project commands are discovered from actual project files
 - [x] Constraints → Strategy → Design → Implementation path is logical
 - [x] Every component in diagram has directory mapping
 - [x] Error handling covers all error types
 - [x] Quality requirements are specific and measurable
 - [x] Component names consistent across diagrams
 - [x] A developer could implement from this design
-

Constraints

CON-1 Target stack is fixed by psg-hub: Next.js 16 App Router, React 19.2, TypeScript strict, Supabase Postgres (shared project `gylkzkmcmbdftxieyabw`), Redis, Tailwind 4 + shadcn, Vercel hosting, Vercel Sandbox for Python workers. No new datastore. The four FileMaker files collapse into the single shared Supabase schema.

CON-2 Delivery is sequential on one team. The customer track (v0.1–v0.4) ships first. The ops backbone that replaces FileMaker ships v1.1 → v1.4. FileMaker stays the authoritative daily driver until the v1.3 cutover and stays read-only for history after. Historical data migration is optional add-on scope v1.3.5 (Q15/Q2).

CON-3 PII is load-bearing. Repair customers are end consumers with names, addresses, phones, emails, and survey content. All new tables follow the existing `psg_sensitive_pii_*` redaction patterns, RLS-clamp to authorized shops, and gate ops tables by `security_profiles.functions_jsonb`. OAuth and vendor secrets are encrypted at rest (pgsodium).

CON-4 No FileMaker event-trigger layer exists to preserve. The DDR analysis confirmed 0 layout triggers, 12 object `OnObjectExit` validations, and 1 in-file PSOS. All automation today is the Web file firing Perform Script on Server on a FileMaker Server schedule, plus date-stamp incremental processing. psg-hub must rebuild automation as Vercel cron + Postgres, not port triggers.

CON-5 Behavior must be preserved exactly where it drives revenue or customer contact: letter eligibility windows, the survey alert classifications (Perfect / Misfire / Hot Spot / Unresolved / Referral), the suppression rule (an open survey alert removes a customer from routine mail), and the "printed/posted" idempotency stamps. These are specified from the DDR, not reinvented.

Implementation Context

IMPORTANT: Read and analyze ALL listed context sources before implementing.

Required Context Sources

Documentation Context

- doc: agents/CIO/FileMaker-Analysis-2026-06-08/03 - Cross-File Architecture Analysis.md
relevance: CRITICAL
why: "Source-of-behavior for the survey-letter feedback loop, match keys, and the four-file topology"
- doc: agents/CIO/FileMaker-Analysis-2026-06-08/01 - Advantage - Business Logic Profile.md
relevance: CRITICAL
why: "Letter lifecycle, print/production pipeline, eligibility windows, the report engine"
- doc: agents/CIO/FileMaker-Analysis-2026-06-08/02 - Survey - Business Logic Profile.md
relevance: CRITICAL
why: "Survey scoring, the Perfect/Misfire/HotSpot/Unresolved/Referral alert subsystem, post-date stamps"
- doc: agents/CIO/FileMaker-Analysis-2026-06-08/04 - Import Flush - Business Logic Profile.md
relevance: HIGH
why: "The 325 per-shop importers that import_templates + psg-import replace"
- doc: agents/CIO/FileMaker-Analysis-2026-06-08/05 - Web - Business Logic Profile.md
relevance: HIGH
why: "The cron/scheduler the Vercel cron surface replaces"
- doc: projects/psg-hub/PLANNING.md
relevance: CRITICAL
why: "71 design decisions, data model, API surface, and v0.1-v2.0 milestones this SDD plugs into"
- doc: apps/psg-hub/.paul/codebase/{ARCHITECTURE,STACK,INTEGRATIONS,CONCERNS}.md
relevance: HIGH
why: "Existing patterns to inherit (Supabase SSR, pg pool, dashboard component pattern, cron gaps)"

Code Context

- file: apps/psg-hub/src/app/api/
relevance: HIGH
why: "REST route conventions for the new ops + cron endpoints"
- file: apps/psg-hub/supabase/migrations/
relevance: CRITICAL
why: "Where the new schema migrations land; resolve against BSM + advantage-portal schemas"
- file: psg-import/src/
relevance: HIGH
why: "R0/Estimate import preprocessor absorbed into v1.1 ops; replaces FileMaker Import Flush"

```

- file: psg-data-lake/Export/
  relevance: HIGH
  why: "Vendor xlsx (Advantage repair + survey responses) – a migration source alongside the DDR CSVs"

- file: psg-advantage-portal/src/lib/{supabase,postgres}/
  relevance: MEDIUM
  why: "SSR client + pg pool patterns to reuse for heavy report reads"

```

External APIs

```

- service: SendGrid
  relevance: HIGH
  why: "Transactional + bulk letter email; replaces Advantage Email Report delivery on psgweb.me"

- service: Lob.com
  relevance: HIGH
  why: "Print-and-mail vendor for physical letters (Production dual adapter, primary path)"

- service: Twilio
  relevance: MEDIUM
  why: "SMS survey reminders + production status (new capability, not in FileMaker)"

- service: FileMaker Data API (psgweb.me host)
  relevance: MEDIUM
  why: "Optional read bridge during parallel run (v1.1–v1.3) and the v1.3.5 migration extract path"

```

Implementation Boundaries

- **Must Preserve:** letter eligibility windows (3/6/12/18/24 month, birthday, drivers license); survey alert classifications and thresholds; the suppression rule (open alert removes a customer from routine mail); idempotent "printed/posted" semantics; per-shop and per-insurer letter variants; the customer↔survey match relationship.
- **Can Modify:** the composite string match key (replace with a real foreign key); the 325 per-shop import scripts (replace with `import_templates`); the three live alert-script generations (collapse to one); the per-letter triplicate scripts (Preview/Print/Loop) collapse into batch + template + status.
- **Must Not Touch:** the live FileMaker files during parallel run, except read-only extracts. psg-hub writes its own data; no two-way sync back into FileMaker. BSM and advantage-portal shipped schemas are extended, not rebuilt.

External Interfaces

System Context Diagram

(diagram in source markdown; see architecture diagram page)

Interface Specifications

```

inbound:
- name: "Ops Web UI"
  type: HTTPS
  format: REST (Next.js route handlers)
  authentication: Supabase Auth session + security_profiles function flags

```

```

data_flow: "Account managers manage companies, ROs, estimates, surveys, production batches"

- name: "Mail vendor webhook (Lob)"
  type: HTTPS
  format: JSON
  authentication: HMAC signature
  data_flow: "Delivery status updates for mailed letters → mail_vendor_jobs"

- name: "Vercel Cron"
  type: HTTPS (scheduled)
  format: internal
  authentication: cron secret header
  data_flow: "Eligibility compute, survey ingest, batch status poll, alert classification"

outbound:
- name: "SendGrid"
  type: HTTPS
  format: REST
  authentication: API Key
  data_flow: "Letter and report email"
  criticality: HIGH

- name: "Lob.com"
  type: HTTPS
  format: REST
  authentication: API Key
  data_flow: "Physical letter print + mail"
  criticality: HIGH

- name: "Twilio"
  type: HTTPS
  format: REST
  authentication: API Key
  data_flow: "Survey reminder + production status SMS"
  criticality: MEDIUM

- name: "FileMaker Data API"
  type: HTTPS
  format: REST
  authentication: token (session)
  data_flow: "Read-only extract during parallel run and v1.3.5 migration"
  criticality: MEDIUM

data:
- name: "Supabase Postgres"
  type: PostgreSQL
  connection: Supabase SSR client + pg pool for heavy report reads
  data_flow: "All ops + survey + production state"

- name: "Redis"
  type: Redis (ioredis)
  connection: Client library
  data_flow: "Rate limit + report cache"

```

Cross-Component Boundaries

- **API Contracts:** `/api/ops/*`, `/api/ops/reports/*`, `/api/cron/*` are internal contracts owned by the ops track. The mail vendor webhook is an external contract that cannot break without a Lob dashboard change.
- **Team Ownership:** single delivery team; ops modules owned by the same team that ships customer track. Python migration worker owned jointly with whoever maintains `psg-data-lake`.

- **Shared Resources:** one Supabase project shared with BSM, advantage-portal, ads-dashboard, local_reach. Schema migrations must not collide.
- **Breaking Change Policy:** ops endpoints are internal; version in-path only if a stable external consumer appears. Migrations are forward-only.

Project Commands

```
# Discovered from apps/psg-hub/package.json + workspace root
Install: pnpm install
Dev:     pnpm dev --filter=psg-hub
Test:    pnpm test          # Vitest 4
E2E:     pnpm --filter=psg-hub exec playwright test
Lint:     pnpm lint
Typecheck: pnpm typecheck
Build:    pnpm build        # turbo

# Database (Supabase)
Migrate: supabase migration up # apply migrations in apps/psg-hub/supabase/migrations/
Seed:    node apps/psg-hub/scripts/seed.mjs # roles, modules, security_profiles, master data

# Migration worker (Python, Vercel Sandbox / psg-data-lake venv)
Extract+Load: python psg-data-lake/fm_migrate.py --entity repair_customers # NEW (v1.3.5)
```

Solution Strategy

- **Architecture Pattern:** Module-per-domain inside the psg-hub Next.js App Router, over the shared Supabase schema, with scheduled work moved to Vercel cron and heavy/batch work to a Python worker in Vercel Sandbox. The FileMaker four-file split disappears; the files become logical modules (Ops, Surveys, Production, Reports) on one database.
- **Integration Approach:** Re-platform behavior, do not port code. Each FileMaker subsystem maps to an already-planned psg-hub module and table set. The DDR analysis supplies the exact rules (eligibility windows, alert thresholds, suppression, idempotency) those modules must encode. Historical data migration is a separate, optional phase (v1.3.5) so functional replacement is not blocked by data movement.
- **Justification:** psg-hub PLANNING.md already commits to replacing FileMaker and already drafts the target tables (`repair_customers` , `survey_responses` , `production_batches` , the 26 reports) and cron surfaces. This SDD fills the one gap PLANNING flagged: the behavioral spec for what those modules must do. Re-platforming on the planned schema is lower risk than a FileMaker-to-SQL lift-and-shift, because the analysis showed the FileMaker design carries heavy duplication (325 import scripts, triplicate letter scripts, three alert generations) that should not be reproduced.
- **Key Decisions:** (1) composite string match key becomes a real foreign key; (2) date-window eligibility finds become SQL + cron; (3) date-stamp idempotency becomes status columns + watermarks; (4) the Web PSOS scheduler becomes Vercel cron; (5) historical migration is ETL through `psg-data-lake` , gated behind v1.3.5.

Building Block View

Components

(diagram in source markdown; see architecture diagram page)

Directory Map

Component: psg-hub ops + surveys + production + reports

```
apps/psg-hub/
├── src/
│   ├── app/
│   │   ├── (ops)/ops/
│   │   │   ├── companies/           # NEW: company/shop ops (v1.1)
│   │   │   ├── repair-customers/    # NEW: end-consumer records (v1.1)
│   │   │   ├── repair-orders/      # NEW: R0 entry + import (v1.1)
│   │   │   ├── estimates/          # NEW: estimate entry + import (v1.1)
│   │   │   ├── surveys/            # NEW: survey entry + view (v1.1)
│   │   │   ├── production/         # NEW: batch build + print queue (v1.3)
│   │   │   └── reports/            # NEW: 26 reports (v1.4)
│   │   └── api/
│   │       ├── ops/                # NEW: ops CRUD + import endpoints
│   │       ├── ops/reports/        # NEW: report query endpoints
│   │       ├── cron/               # NEW: eligibility, survey-ingest, batch-status
│   │       └── webhooks/lob/       # NEW: mail vendor status
│   └── lib/
│       ├── eligibility/            # NEW: letter eligibility windows + suppression rule
│       ├── surveys/scoring/        # NEW: Perfect/Misfire/HotSpot/Unresolved/Referral
│       ├── production/             # NEW: batch + print id + idempotency
│       ├── reports/                # NEW: report engine (replaces PDF Generate/Email Report)
│       ├── filemaker/              # NEW: Data API read client (parallel-run + migration)
│       └── supabase/migrations/    # NEW: ops backbone + surveys + production + reports
```

Component: migration worker (v1.3.5)

```
psg-data-lake/
├── fm_migrate.py           # NEW: orchestrator (extract → transform → load)
├── fm_extract/             # NEW: DDR CSV + FileMaker Data API readers
├── fm_transform/           # NEW: match-key resolution, PII redaction, dedupe
└── tests/test_fm_migrate.py # NEW
```

Interface Specifications

Data Storage Changes

Most target tables already exist in PLANNING.md. This SDD adds the behavioral columns the DDR analysis proved are required, and one cross-cutting addition: the survey↔customer link becomes a real foreign key.

```
Table: survey_responses (EXTEND - planned in PLANNING.md)
  ADD COLUMN: repair_customer_id (fk repair_customers.id, nullable until matched) # replaces
composite string match key
  ADD COLUMN: alert_class (enum: perfect|misfire|hotspot|unresolved|referral|none) # from DDR
classification calcs
  ADD COLUMN: csi_resolve (numeric) # SQ_Scale_CSI_Resolve
  ADD COLUMN: would_recommend (bool) # SQ_Would_Recom_Facility
  ADD COLUMN: unresolved_shop (bool) # SQ_Unresolved_Shop
  ADD COLUMN: referral_consumer (bool) # SQ_Referral_Consumer
  ADD COLUMN: alert_posted_at (timestampz, null) # replaces SQ_Alert_PostDate_* idempotency
stamps
  ADD COLUMN: referral_letter_posted_at (timestampz, null) # replaces S_RC_Referral_Letter_Post
  ADD INDEX: idx_survey_match (repair_customer_id, submitted_at)

Table: repair_customers (EXTEND - planned)
  ADD COLUMN: legacy_match_key (text, indexed) # RC_MatchField_Master, kept for migration
```

```

reconciliation only
  ADD COLUMN: referral_tracking_enabled (bool)      # Master Client::M_ReferralNoted_flag (per-
company, see company_programs)
  ADD COLUMN: credit_hold (bool)                    # M_CreditHold_flag (suppresses referral path)

Table: letter_eligibility (NEW)
  id: primary_key
  repair_customer_id: fk repair_customers.id
  letter_kind: enum (three_month|six_month|one_year|eighteen_month|two_year|birthday|
drivers_license|thank_you|referral)
  eligible: bool
  computed_at: timestamptz
  suppressed_by_alert: bool                        # the survey-alert suppression rule, made
explicit
  printed_at: timestamptz, null                    # replaces Letter*_Printed stamps (idempotency)
  UNIQUE (repair_customer_id, letter_kind, period_key) # period_key = cycle bucket, prevents
double-send

Table: company_programs (EXTEND - planned)
  ADD COLUMN: referral_tracking_enabled (bool)      # drives the Referral vs Misfire branch per shop
# customizations_jsonb already holds logo/header/footer/greeting (replaces Insert_Letter Header
scripts)

# production_batches, production_documents, production_reprint_log, mail_vendor_jobs,
# email_jobs, sms_jobs, import_templates, security_profiles already defined in PLANNING.md – reuse
as-is.

schema_doc: projects/psg-hub/PLANNING.md (Data Model)
migration_scripts: apps/psg-hub/supabase/migrations/

```

Internal API Changes

```

Endpoint: Compute letter eligibility (cron)
Method: POST
Path: /api/cron/eligibility-compute
Request: { letter_kind?: string } # all kinds if omitted
Behavior:
  - Find repair_customers in the date window for each letter_kind (replaces SS - Eligibility - *
finds)
  - Apply suppression: WHERE NOT EXISTS open survey alert (alert_posted_at within window) # the
DDR rule
  - Upsert letter_eligibility rows (idempotent on repair_customer_id+letter_kind+period_key)
Response: { computed: int, eligible: int, suppressed: int }

Endpoint: Ingest + classify surveys (cron)
Method: POST
Path: /api/cron/survey-ingest
Behavior:
  - Pull new survey_responses since watermark
  - Resolve repair_customer_id (FK match; fall back to legacy_match_key during migration)
  - Compute alert_class from csi_resolve + would_recommend + unresolved_shop + referral gates #
DDR calcs
Response: { ingested: int, classified: {perfect,misfire,hotspot,unresolved,referral,none} }

Endpoint: Build production batch
Method: POST
Path: /api/ops/production/batches
Request: { name: string (unique), product_ids: [], company_ids: []|null }
Response: { batch_id, document_count }

Endpoint: Mark documents printed (idempotent)

```

```
Method: POST
Path: /api/ops/production/batches/{id}/print
Behavior: set production_documents.status=printed, printed_at=now() only where status=unprinted
Response: { printed: int, skipped_already_printed: int }
```

Endpoint: Lob status webhook

```
Method: POST
Path: /api/webhooks/lob
Request: Lob event JSON (HMAC verified)
Response: 200; updates mail_vendor_jobs.status
```

Endpoint: Run report

```
Method: GET
Path: /api/ops/reports/{report_slug}
Behavior: parameter-driven query (replaces PDF Generate Report(ReportName)); 26 slugs from PLANNING.md
Response: { rows, meta } or rendered PDF/email job id
```

Application Data Models

ENTITY: SurveyResponse (EXTENDED)

FIELDS:

```
repair_customer_id: fk (NEW - real FK replaces composite match key)
alert_class: enum (NEW)
csi_resolve, would_recommend, unresolved_shop, referral_consumer (NEW)
alert_posted_at, referral_letter_posted_at (NEW - idempotency)
```

BEHAVIORS:

```
classify(): alert_class      # NEW - encodes DDR rules below
isOpenAlert(asOf): bool      # NEW - drives suppression
```

ENTITY: LetterEligibility (NEW)

FIELDS: repair_customer_id, letter_kind, eligible, suppressed_by_alert, printed_at, period_key

BEHAVIORS:

```
compute(window, suppressionRule): void # replaces SS - Eligibility - * scripts
markPrinted(): void                    # replaces Loop - * Printed date stamps
```

ENTITY: ProductionBatch / ProductionDocument (NEW - per PLANNING)

BEHAVIORS:

```
build(productIds, companyIds): documents # replaces Print - * production scripts
print(): idempotent status transition    # replaces date-stamp re-send guard
```

Integration Points

```
- from: cron/survey-ingest
  to: lib/surveys/scoring
  protocol: in-process
  data_flow: "raw survey rows → alert_class + posted stamps"

- from: cron/eligibility-compute
  to: lib/eligibility
  protocol: in-process
  data_flow: "date windows + open-alert suppression → letter_eligibility"

- from: ops/production
  to: Lob.com / SendGrid
  protocol: REST
  data_flow: "batch documents → printed/mailed/emailed; webhook returns status"
```

External_Service_FileMaker:

```
- integration: "Read-only Data API extract during parallel run + v1.3.5 migration"
- critical_data: [repair customers, R0s, estimates, survey responses, production history]
```

Implementation Examples

Example: Survey alert classification (the DDR rules, made executable)

Why this example: This is the load-bearing logic of the whole customer-experience loop. The thresholds come straight from the Survey file calcs and must match exactly.

```
// Mirrors Survey Input SQ Alert *_post calcs (see Survey profile $3).
function classifySurvey(s: SurveyResponse, company: CompanyProgram): AlertClass {
  const happy = s.wouldRecommend && !s.unresolvedShop;
  if (happy && s.referralConsumer && company.referralTrackingEnabled && !s.creditHold) {
    return "referral";          // SQ_Alert_Referral_post
  }
  if (happy && s.csiResolve === 1) return "perfect";    // SQ_Alert_Perfect_post
  if (happy && s.csiResolve < 1) return "misfire";      // satisfied, would-recommend, not perfect =
  missed referral
  if (s.unresolvedShop)          return "unresolved";
  if (s.csiResolve < 1)          return "hotspot";
  return "none";
}
```

Example: Eligibility suppression rule

Why this example: The DDR proved an open survey alert silently removes a customer from routine mail. Make it explicit and queryable, not an emergent side effect.

```
-- Replaces Advantage "SS - Eligibility - 3Month" date-window find + IsEmpty(survey alert) guard.
INSERT INTO letter_eligibility (repair_customer_id, letter_kind, eligible, suppressed_by_alert,
period_key, computed_at)
SELECT rc.id, 'three_month',
       (sr.id IS NULL),          -- eligible only if no open alert
       (sr.id IS NOT NULL),      -- record why suppressed
       date_trunc('month', rc.ro_completed_at),
       now()
FROM repair_customers rc
LEFT JOIN survey_responses sr
  ON sr.repair_customer_id = rc.id
 AND sr.alert_class <> 'none'
 AND sr.alert_posted_at >= now() - interval '120 days'
WHERE rc.ro_completed_at BETWEEN now() - interval '120 days' AND now() - interval '90 days'
ON CONFLICT (repair_customer_id, letter_kind, period_key) DO NOTHING; -- idempotent, no double-
send
```

Runtime View

Primary Flow: nightly letter cycle

1. Vercel cron hits `/api/cron/survey-ingest`. New surveys are matched to repair customers by FK and classified into an `alert_class`; `alert_posted_at` is stamped.
2. Vercel cron hits `/api/cron/eligibility-compute`. For each letter kind, customers in the date window are found, customers with an open alert are suppressed, and `letter_eligibility` rows are upserted idempotently.

3. An account manager (or a scheduled batch) builds a production batch from eligible customers. Documents get a unique print id.
4. Documents are printed via Lob (physical) or emailed via SendGrid, then marked `printed / mailed`. Re-running skips already-printed documents.
5. Lob webhook updates delivery status on `mail_vendor_jobs`.

(diagram in source markdown; see architecture diagram page)

Error Handling

- Unmatched survey (no `repair_customer_id`): keep row, leave FK null, surface in a "needs match" ops queue; never silently drop. Replaces the FileMaker risk where a shop-id/month/year mismatch broke the link invisibly.
- Vendor (Lob/SendGrid) failure: mark `mail_vendor_jobs / email_jobs` failed with error; cron retries with backoff; document stays `unprinted` so it is not lost.
- Duplicate run / re-trigger: idempotent upserts and `printed_at` guards prevent double-send. This replaces the date-stamp pattern.
- Cron missed (the single-point-of-automation risk from the Web file): each cron is idempotent and window-based, so a missed night self-heals on the next run; add a freshness alert if `computed_at` is stale > 36h.

Complex Logic: incremental, idempotent processing

```
ALGORITHM: Incremental cycle (replaces FileMaker date-stamp processing)
INPUT: watermark (last run), date window
1. SELECT rows changed/created since watermark
2. RESOLVE foreign keys (repair_customer_id); queue unmatched
3. CLASSIFY / COMPUTE (alert_class or eligibility)
4. UPSERT with unique constraint (idempotent)
5. ADVANCE watermark
6. EMIT counts for observability
```

Deployment View

Single Application Deployment

- **Environment:** Vercel (existing psg-advantage-portal project re-linked + renamed psg-hub). Cron via Vercel Cron. Python migration worker via Vercel Sandbox or the `psg-data-lake` venv.
- **Configuration:** `SENDGRID_API_KEY`, `LOB_API_KEY`, `LOB_WEBHOOK_SECRET`, `TWILIO_*`, `CRON_SECRET`, `FILEMAKER_DATA_API_*` (parallel run + migration), existing `SUPABASE_*` + `REDIS_URL`.
- **Dependencies:** Supabase project `gylkzmcmbdftxieyabw`; SendGrid sender auth on `psgweb.me` (SPF/DKIM/DMARC); Lob account; Twilio number.
- **Performance:** LCP < 3s on `/ops/*` and `/ops/reports/*` (PLANNING quality bar). Heavy reports use the `pg` pool + Redis cache, not PostgREST.

Multi-Component Coordination

- **Deployment Order:** schema migration → ops endpoints → cron → UI. Production module (v1.3) requires Lob/SendGrid live (v0.1 setup).
- **Feature Flags:** gate `/ops/*` and each cron behind the `modules` registry + `security_profiles` so parallel run can be toggled per function.

- **Rollback Strategy:** forward-only migrations; new tables are additive, so a UI rollback leaves data intact. FileMaker remains authoritative until v1.3 cutover, so rollback = point users back to FileMaker.
- **Data Migration Sequencing (v1.3.5):** master data → companies/employees → repair_customers → ROs/estimates → survey_responses → production history. Each step reconciles counts against the DDR (281K repair customers, 334K surveys) before the next.

Cross-Cutting Concepts

Pattern Documentation

```
- pattern: Supabase SSR client + pg pool (advantage-portal)
  relevance: CRITICAL
  why: "Reuse for ops reads and heavy report queries"
- pattern: Vercel cron + idempotent watermark worker (NEW)
  relevance: CRITICAL
  why: "Replaces the Web PSOS scheduler and FileMaker date-stamp incrementality"
- pattern: import_templates field mapping (NEW, absorbs psg-import)
  relevance: HIGH
  why: "Replaces 325 per-shop FileMaker import scripts with one mapping-driven importer"
```

System-Wide Patterns

- Security: Supabase Auth + RBAC roles + `security_profiles.functions_jsonb (manage_companies, manage_surveys, manage_production, manage_reports, manage_data_imports)`. RLS clamps customer data to authorized shops.
- Error Handling: never drop unmatched records; queue them. Idempotent writes everywhere a FileMaker date-stamp used to guard.
- Performance: Redis cache for the 26 reports; `pg pool` for heavy reads; cron windows bounded.
- Logging/Auditing: `access_audit` (append-only), `python_worker_jobs`, `email_jobs / sms_jobs / mail_vendor_jobs` give the audit trail FileMaker lacked.
- PII: `psg_sensitive_pii_*` redaction patterns; survey `raw_payload` redacted; pgsodium for secrets.

UI (ops)

- Information Architecture: `/ops/{companies,repair-customers,repair-orders,estimates,surveys,production,reports}` behind the ops shell; reuse the advantage-portal dashboard component pattern (page → Dashboard → Panel + Metric + chart).
- Design System: `packages/ui` PSG brand tokens + shadcn; no per-screen styling.
- Interaction: optimistic updates for ops CRUD; explicit confirm + dry-run for production batch print (irreversible mail spend).

Architecture Decisions

- [] ADR-1 Re-platform behavior, do not port FileMaker code or schema 1:1: rebuild on the planned psg-hub schema.
- Rationale: the DDR showed heavy duplication (325 import scripts, triplicate letter scripts, three alert generations) that should not be reproduced; PLANNING already drafts the target tables.
- Trade-offs: more upfront design than a lift-and-shift; requires the DDR as the behavioral source of truth.
- User confirmed: *Pending*

- [] ADR-2 Replace the composite string match key with a real foreign key (`survey_responses.repair_customer_id`).
- Rationale: the FileMaker shop-id+month+year string key broke links silently; a FK plus a "needs match" queue is safer and queryable.
- Trade-offs: migration must resolve legacy keys to FKs; keep `legacy_match_key` for reconciliation.
- User confirmed: *Pending*
- [] ADR-3 Move all automation to Vercel cron + idempotent Postgres workers; retire the Web PSOS scheduler and date-stamp pattern.
- Rationale: no FileMaker triggers to preserve; cron + unique constraints give idempotency and self-healing.
- Trade-offs: a missed cron must self-heal by window design; add freshness alerting.
- User confirmed: *Pending*
- [] ADR-4 Make the survey-alert suppression rule explicit in `letter_eligibility` (`suppressed_by_alert`) rather than an emergent join side effect.
- Rationale: the suppression was invisible in FileMaker; surfacing it prevents accidental mailing to unhappy customers and is auditable.
- Trade-offs: one extra column + compute step.
- User confirmed: *Pending*
- [] ADR-5 Historical data migration is optional v1.3.5 via a `psg-data-lake` Python worker, sourced from DDR CSV exports plus the FileMaker Data API; functional replacement (v1.1–v1.4) does not depend on it.
- Rationale: matches PLANNING Q2/Q15; lets FileMaker stay read-only for history without blocking cutover.
- Trade-offs: a period where current data is in psg-hub and deep history is in read-only FileMaker.
- User confirmed: *Pending*

Quality Requirements

- Performance: `/ops/*` and `/ops/reports/*` LCP < 3s; eligibility + survey-ingest cron each complete < 5 min over full population; report queries cached in Redis.
- Reliability: every cron idempotent; zero double-sends verified by unique constraints; unmatched-survey rate visible and < 2% after migration reconciliation.
- Security: ops tables RLS-gated by security profile flags; customer data clamped to authorized shops; PII redaction parity with existing patterns; manual PII sign-off at v1.1, v1.3, v2.0.
- Migration integrity (v1.3.5): post-load counts reconcile to DDR within tolerance (`repair_customers` ≈ 281K, `survey_responses` ≈ 334K); spot-check 50 customer↔survey links resolve correctly.

Acceptance Criteria

Surveys + alerts (PLANNING v1.1/v1.4) - [] WHEN a survey is ingested, THE SYSTEM SHALL resolve its `repair_customer_id` and set `alert_class` using the DDR classification rules. - [] IF a survey cannot be matched to a repair customer, THEN THE SYSTEM SHALL retain it and surface it in a needs-match queue.

Eligibility + suppression (v1.3) - [] WHEN eligibility is computed, THE SYSTEM SHALL exclude any customer with an open survey alert in the window and record `suppressed_by_alert`. - [] THE SYSTEM SHALL never create a duplicate letter for the same customer, letter_kind, and period.

Production (v1.3) - [] WHEN a batch is printed, THE SYSTEM SHALL transition only `unprinted` documents to `printed` and stamp `printed_at`. - [] WHEN Lob returns a status, THE SYSTEM SHALL update the matching `mail_vendor_jobs` row.

Reports (v1.4) - [] THE SYSTEM SHALL serve all 26 reports (incl. Perfect Score, Mis-Fire, Hot Spot, Unresolved Issue, Referral Noted) from a parameter-driven engine.

Migration (v1.3.5, optional) - [] WHERE the migration add-on is enabled, THE SYSTEM SHALL load historical entities and reconcile counts to the DDR before cutover.

Risks and Technical Debt

Known Technical Issues (inherited from FileMaker, per DDR)

- Stale external references in the FileMaker files (BridgeSystem hardcoded path, EmployeeSatisfaction, Import Flush2, dormant PSG_SurveyExport_07.1, broken "Phoenix Solutions Group"). Do not recreate these dependencies; the migration extract must tolerate their absence.
- Three live generations of survey alert scripts (v2/WIP/v3 LIVE). Confirm the v3 LIVE logic is authoritative before encoding `classifySurvey`.
- Import sources include machine-specific local paths. The migration must source from the DDR exports and the Data API, not from a developer workstation.

Technical Debt to Avoid Reproducing

- The 325 per-shop importers → one `import_templates`-driven importer.
- Triplicate per-letter scripts (Preview/Print/Loop) → batch + template + status.
- Composite string match key → foreign key.

Implementation Gotchas

- The `referral` vs `misfire` branch depends on per-company `referral_tracking_enabled` and `credit_hold`; both were read cross-file in FileMaker. Seed them onto `company_programs / repair_customers` before classification runs, or every survey misclassifies.
- Eligibility windows use RO completion date, not survey date. Keep the two date axes distinct.
- Parallel run (v1.1–v1.3): FileMaker stays authoritative. Do not write back to FileMaker; psg-hub is downstream only until cutover.

Glossary

Domain Terms

Term	Definition	Context
Misfire	A satisfied, would-recommend customer who did not score a perfect CSI; a missed referral	Survey alert class; drives Mis-Fire report + alert
Perfect Score	Survey with CSI resolve = 1 and would-recommend	Drives Perfect Score report + referral path
Hot Spot	A negative survey result needing follow-up	Survey alert class

Term	Definition	Context
Suppression	Removing a customer from routine mail because an open survey alert exists	Core feedback-loop rule (ADR-4)
Production batch	A set of letters generated for printing/mailing	Replaces FileMaker Print/Production scripts

Technical Terms

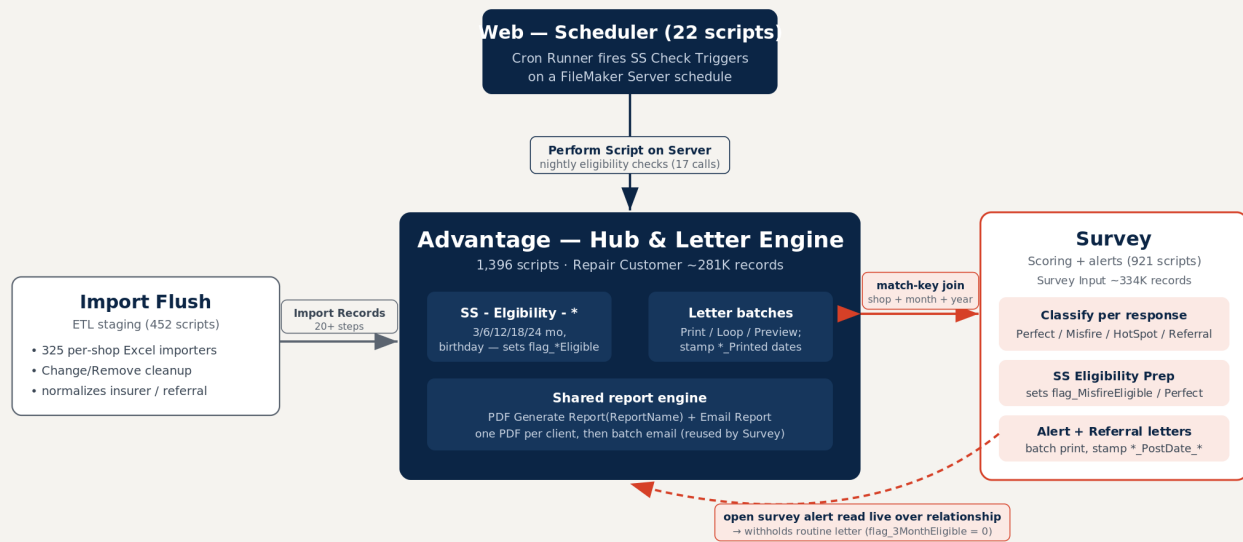
Term	Definition	Context
DDR	FileMaker Database Design Report	Source of all behavioral rules in this SDD
Idempotent cycle	A run that can repeat without duplicate effects	Replaces FileMaker date-stamp processing
Parallel run	Period where FileMaker and psg-hub both operate, FileMaker authoritative	v1.1–v1.3
Cutover	The point psg-hub replaces FileMaker as daily driver	v1.3

API/Interface Terms

Term	Definition	Context
Report engine	Parameter-driven report runner (<code>/api/ops/reports/{slug}</code>)	Replaces PDF Generate Report(ReportName) + Email Report
Mail vendor adapter	Lob (physical) + in-house/SendGrid (email) dual path	<code>mail_vendor_jobs</code> , <code>email_jobs</code>
Needs-match queue	Surface for surveys without a resolved <code>repair_customer_id</code>	Error handling for FK resolution

PhoenixSolutions FileMaker — System Architecture

Four-file, relationally interconnected solution, hub-and-spoke around Advantage. DDR dated 5/27/2025.



The survey → letter feedback loop (automated linkage, batch-run output)

1. Survey scored & classified → 2. Eligibility flags set (server-side) → 3. Advantage eligibility reads alert state over the match-key join
 4. Customers with open alerts are auto-suppressed from routine mailings → 5. Misfire / Perfect / Referral letters produced as batch runs, post-dates stamped
- No in-file script triggers anywhere (0 layout, 12 object OnObjectExit). Automation = Web server schedule + date-stamp incremental processing.

Legend: navy arrow = server-side script call · gray arrow = data import · **red** = cross-file relationship / survey-driven suppression · dashed = read-only relationship

External (out of scope): EmployeeSatisfaction, PSG_SurveyExport_07.1 (dormant), BridgeSystem, Import Flush2, "Phoenix Solutions Group" (broken).

Counts reconcile with Summary.xml. TO/relationship field pairs reconstructed from match-key calcs (DDR exported predicate counts only).

Implementation Plan

This plan turns the SDD (`solution-design.md`) into a buildable sequence for the ops track. It re-platforms the four FileMaker files (Advantage, Survey, Import Flush, Web) into psg-hub modules on the shared Supabase schema, replaces the Web PSOS scheduler with Vercel cron, and replaces the date-stamp idempotency pattern with status columns plus unique constraints.

Scope here is the ops track only (v1.1 to v1.4 plus optional v1.3.5). The customer track (v0.1 to v0.4) ships first and is out of scope for this plan. FileMaker stays the authoritative daily driver through the v1.3 cutover and stays read-only for history after.

The behavioral source of truth is the DDR cross-file analysis. Where this plan names rules (eligibility windows, alert classes, suppression, idempotency), they come from the analysis, not from new design.

Components and Dependencies

The build has six logical components. Each maps to a directory and a milestone.

Component	Directory	Replaces (FileMaker)	Depends on
Schema + RBAC/ security_profiles	<code>supabase/migrations/</code> , <code>scripts/seed.mjs</code>	the four-file split, no FM security layer	nothing (foundation)
Ops CRUD endpoints + UI	<code>src/app/api/ops/</code> , <code>src/app/ops/ops/</code>	Advantage data-entry layouts, Import Flush	schema, RBAC
Cron workers	<code>src/app/api/cron/</code>	Web PSOS scheduler, date-stamp processing	schema, scoring service, eligibility service
Survey scoring service	<code>src/lib/surveys/scoring/</code>	Survey <code>SQ_Alert_*</code> classification calcs	schema, <code>company_programs</code> seeds
Eligibility service	<code>src/lib/eligibility/</code>	Advantage <code>SS - Eligibility - * finds + suppression</code>	schema, scoring (open-alert reads)
Production batch + print	<code>src/lib/production/</code> , <code>src/app/api/ops/production/</code>	Advantage Print/Loop letter batches	schema, eligibility, mail adapters
Report engine	<code>src/lib/reports/</code> , <code>src/app/api/ops/reports/</code>	Advantage PDF Generate Report + Email Report	schema, all upstream data populated
Migration worker (optional)	<code>psg-data-lake/fm_migrate.py</code>	one-time history move	schema fully landed, all services stable

Dependency Notes

- The schema delta is the gate for everything. No service compiles without `letter_eligibility` , the `survey_responses` columns, and the `repair_customers / company_programs` columns.
- RBAC and `security_profiles` ship with the schema because every ops endpoint and cron is gated by `security_profiles.functions_jsonb` flags (`manage_companies` , `manage_surveys` , `manage_production` , `manage_reports` , `manage_data_imports`).
- The eligibility service reads open-alert state, so survey ingest and classification must produce `alert_class` and `alert_posted_at` before eligibility runs are meaningful.
- The production batch builds from eligible customers, so eligibility must be computing before a batch is useful (though batch CRUD itself only needs schema).

- The report engine reads every other table, so reports land last among the functional modules.
- The migration worker is the only Python component and the only optional one; it never blocks functional replacement (ADR-5).

Implementation Order

Build in this order. Each stage is a hard prerequisite for the next unless marked parallel.

1. **Schema + RBAC/security_profiles.** Migrations for every table delta the SDD lists, plus RBAC seeds. This is the foundation; nothing else starts until migrations apply cleanly and seeds load.
2. **Ops CRUD.** Companies, employees, programs, repair_customers, repair_orders, estimates, surveys (entry/view), and master data (products, items, vehicles, insurance_companies, insurance_agents). Plus the `import_templates` importer that absorbs psg-import and retires the 325 per-shop scripts.
3. **Survey ingest + classify.** The scoring service (`classifySurvey`) and the `/api/cron/survey-ingest` worker. FK resolution to `repair_customer_id` , the needs-match queue for unmatched rows, and `alert_class` + `alert_posted_at` stamping.
4. **Eligibility.** The eligibility service (date windows on RO completion date) plus the suppression rule (open survey alert removes a customer from routine mail, recorded as `suppressed_by_alert`), behind `/api/cron/eligibility-compute` . Idempotent upserts on (`repair_customer_id` , `letter_kind` , `period_key`) .
5. **Production.** Batch build, idempotent print (only `unprinted` to `printed`), reprint with audit, and the Lob webhook plus the dual mail adapter (Lob primary, in-house fallback).
6. **Reports.** The parameter-driven report engine and the 26 named reports, starting with one survey-alert report end to end (Perfect Score) to prove the framework.
7. **Migration (optional, v1.3.5).** The `psg-data-lake` Python worker: extract from DDR CSVs plus the FileMaker Data API, resolve legacy match keys to FKs, redact PII, dedupe, load, and reconcile counts.

Why This Order

The order follows data flow. A survey must be scored before its alert can suppress a letter; a letter must be eligible before it can be batched; a batch must exist before a report can recap it. Building against the grain (reports before data, eligibility before scoring) would force stubbing the exact rules the DDR says must be exact.

Mapping to psg-hub Milestones

The PLANNING.md milestone numbers govern. Note the customer track (v0.1 to v0.4) ships first; this is the ops track.

Milestone	Scope from this plan	PLANNING reference
v1.1 Ops Foundation	Schema + RBAC/security_profiles; ops CRUD; master data; <code>import_templates</code> importer; survey entry/view; survey ingest + classify; eligibility service + suppression	PLANNING v1.1 phases 1 to 8
v1.3 Production + cutover	Mail dual adapter; production batches; idempotent print; reprint; Lob webhook; historical search; FileMaker daily-driver cutover	PLANNING v1.3 phases 1 to 6
v1.4 Reports	Report engine framework; the 26 named reports in 5 batches	PLANNING v1.4 phases 1 to 5
v1.3.5 Migration (optional)	<code>psg-data-lake</code> Python worker; historical extract, transform, load, reconcile	PLANNING Decision 32, Q15; SDD ADR-5

Notes on milestone placement:

- The SDD draws eligibility/suppression acceptance criteria under v1.3, but the eligibility service and the survey scoring it depends on are built in v1.1 (PLANNING v1.1 phase 4 covers surveys). This plan builds eligibility + scoring in v1.1 so the suppression rule is provable before the v1.3 production cutover relies on it. v1.3 then consumes eligible customers; it does not rebuild eligibility.
- v1.2 (Ads Mutation Studio) sits between v1.1 and v1.3 in PLANNING but is out of scope for this FileMaker integration. The ops backbone built in v1.1 carries forward unchanged.
- v1.3.5 is sequenced after v1.3 cutover and is gated behind the optional add-on decision. Functional replacement (v1.1 to v1.4) never depends on it.

Parallel vs Sequential Work

Within the single delivery team, most of this is sequential because of the data-flow dependency chain. The parallelizable seams:

Can run in parallel:

- Ops CRUD modules are mutually independent once the schema lands. Companies/employees/programs, repair_customers/ROs, estimates, surveys-entry, and master data CRUD can be split across people if capacity allows.
- The `import_templates` importer is independent of the survey/eligibility chain; it shares only the schema.
- Mail adapter scaffolding (Lob client, in-house adapter, the `MailAdapter` interface) can be built while production batch CRUD is in progress, since the interface is the contract.
- Report SQL for reports that read only already-populated tables (volume/invoicing reports) can be drafted while the survey-dependent reports wait on classification data.
- The migration worker's extract/transform code can be written against DDR CSV samples at any point after the schema is fixed, independent of the live app.

Must be sequential:

- Schema before any service. Hard gate.
- Survey scoring before eligibility (eligibility reads open-alert state).
- Eligibility before a useful production batch (batch builds from eligible customers).
- Data populated before reports that recap it.
- v1.3 production cutover before v1.3.5 migration (migration loads history behind the daily driver).

Verification Checkpoints

Each phase is done when its checkpoint passes. These prove the phase, not just that code exists.

After Schema + RBAC

- `supabase migration up` applies all deltas with no error; `list_tables` shows `letter_eligibility` plus the new columns on `survey_responses`, `repair_customers`, `company_programs`.
- `node scripts/seed.mjs` seeds roles, modules, and the built-in Administrator `security_profile` granting all function flags.
- A `psg_internal` user without `manage_companies` is denied on a guarded ops route; with it, allowed. RLS clamps customer data to authorized shops.

After Ops CRUD

- An account manager creates a company, adds employees, enrolls a program with logo/header/footer overrides, and the records persist with RLS enforced.
- An RO import through an `import_templates` mapping validates against the template, loads rows, and surfaces row-level validation errors. The 325-per-shop pattern is not reproduced; one mapping-driven importer handles all shops.

After Survey Ingest + Classify

- A seeded survey with known scores classifies to the expected `alert_class` per the DDR rules (verify all six: perfect, misfire, hotspot, unresolved, referral, none).
- A survey with no resolvable `repair_customer_id` is retained with a null FK and appears in the needs-match queue; it is never dropped.
- `alert_posted_at` stamps on classification; re-running ingest does not reclassify or double-stamp already-processed rows (watermark + idempotent).

After Eligibility

- For a customer in the 3-month window on RO completion date with no open alert, an eligible `letter_eligibility` row exists.
- For the same customer with an open survey alert in the window, the row records `eligible=false` and `suppressed_by_alert=true`. The suppression is queryable, not emergent.
- Re-running `eligibility-compute` produces no duplicate row for the same `(repair_customer_id, letter_kind, period_key)`.

After Production

- A batch built from eligible customers produces documents each with a unique `print_id`.
- Printing transitions only `unprinted` documents to `printed` and stamps `printed_at`; re-running the print skips already-printed documents (reported as `skipped_already_printed`).
- A reprint writes a `production_reprint_log` row. A Lob webhook (HMAC verified) updates the matching `mail_vendor_jobs` row.

After Reports

- `/api/ops/reports/perfect-score` returns rows matching a hand-checked query over seeded survey data.
- The report framework serves a 12-month date range within the LCP < 3s bar using the `pg` pool plus Redis cache, not PostREST.
- All 26 report slugs resolve (even if some return empty on seed data).

After Migration (optional)

- Post-load counts reconcile to the DDR within tolerance (repair_customers approx 281K, survey_responses approx 334K).
- A spot-check of 50 customer-to-survey links resolves correctly through the new FK (not the legacy match key).
- Unmatched-survey rate is visible and under 2 percent after reconciliation.

Per-Milestone Gate (Playwright)

Each milestone closes with a Playwright happy-path E2E (PLANNING quality gates):

- v1.1: create company to add employees to import RO.
- v1.3: build batch to print to reprint.
- v1.4: open report index to run one report to export.

Risks and Mitigations

Pulled from the SDD Risks section and the DDR analysis.

Risk	Source	Mitigation
Stale FileMaker external references (BridgeSystem hardcoded path, EmployeeSatisfaction, Import Flush2, dormant PSG_SurveyExport_07.1, broken "Phoenix Solutions Group")	DDR sec 6; SDD Known Technical Issues	Do not recreate these dependencies. The migration extract must tolerate their absence and not fail on a missing referenced file. Source only from DDR exports plus the Data API.
Three live generations of alert scripts (v2, WIP, v3 LIVE)	DDR sec 6; SDD Known Technical Issues	Confirm v3 LIVE is the authoritative logic before encoding <code>classifySurvey</code> . Encode exactly one classification path; do not port all three. Verify thresholds against the v3 calcs.
Machine-specific import paths (e.g. <code>/Users/ryan/Downloads/1.fmp12</code> , per-shop Flush files)	DDR sec 6; SDD Known Technical Issues	The importer sources from uploaded files mapped via <code>import_templates</code> , never from a workstation path. The migration worker sources from DDR exports plus the Data API, not a developer machine.
Single point of automation (the Web PSOS scheduler was the only cron)	DDR sec 4; SDD Error Handling	Every cron is idempotent and window-based, so a missed run self-heals on the next run. Add a freshness alert when <code>computed_at</code> is stale beyond 36h. No single script is load-bearing.
Referral flag seeding (the referral vs misfire branch reads per-company <code>referral_tracking_enabled</code> and <code>credit_hold</code> , which were cross-file reads in FileMaker)	DDR sec 3; SDD Implementation Gotchas	Seed <code>company_programs.referral_tracking_enabled</code> and <code>repair_customers.credit_hold</code> before any classification run. Without them every survey misclassifies. Add a seed-completeness check to the survey-ingest preflight.
Composite match key broke links silently (shop-id + month + year string)	DDR sec 3 and 6	Replace with a real FK (<code>survey_responses.repair_customer_id</code> , ADR-2). Keep <code>legacy_match_key</code> for reconciliation only. Unmatched rows go to the needs-match queue, never dropped.
Cleared/back-dated stamps re-fire or skip letters	DDR sec 6	Replace date-stamp guards with status columns plus unique constraints. <code>printed_at</code> guards and <code>ON CONFLICT DO NOTHING</code> make re-runs safe.
Two date axes confused (eligibility uses RO completion date, not survey date)	SDD Implementation Gotchas	Keep the two dates distinct in schema and queries. Eligibility windows key on <code>ro_completed_at</code> ; alert posting keys on survey <code>submitted_at</code> .
Parallel-run write-back (psg-hub must not write into FileMaker)	SDD Implementation Gotchas; CON-2	psg-hub is downstream only until v1.3 cutover. No two-way sync. FileMaker stays authoritative; reads are read-only extracts.

Deployment Order

Per the SDD Deployment View: schema migration, then ops endpoints, then cron, then UI. Production (v1.3) requires Lob/SendGrid live (set up in v0.1). Migrations are forward-only and additive, so a UI rollback leaves data intact; until v1.3 cutover, rollback means pointing users back to FileMaker.

Task Breakdown

Discrete, dependency-ordered tasks grouped by milestone. Paths are under `apps/psg-hub/...`. Terminology matches the SDD exactly (`letter_eligibility`, `alert_class`, `suppressed_by_alert`, `printed_at`, `period_key`, `legacy_match_key`).

Conventions used by every task:

- DB column `profile_id` (not `user_id`).
 - Migrations are forward-only and additive.
 - Every cron is idempotent and window-based.
 - Every ops route is gated by `security_profiles.functions_jsonb`.
 - Project commands: `pnpm typecheck`, `pnpm test` (Vitest 4), `pnpm --filter=psg-hub exec playwright test`, `supabase migration up`, `node scripts/seed.mjs`.
-

Milestone v1.1 — Ops Foundation

Schema and RBAC

- [] Task: Add `letter_eligibility` table migration
- Acceptance: table has `id`, `repair_customer_id` (fk `repair_customers`), `letter_kind` enum (`three_month`, `six_month`, `one_year`, `eighteen_month`, `two_year`, `birthday`, `drivers_license`, `thank_you`, `referral`), `eligible` bool, `suppressed_by_alert` bool, `period_key`, `printed_at` timestampz null, `computed_at`; UNIQUE (`repair_customer_id`, `letter_kind`, `period_key`)
- Verify: `supabase migration up` applies clean; `\d letter_eligibility` shows the unique constraint
- Files: `supabase/migrations/<ts>_create_letter_eligibility.sql`
- [] Task: Extend `survey_responses` with classification + idempotency columns
- Acceptance: adds `repair_customer_id` (fk, nullable), `alert_class` enum (`perfect`, `misfire`, `hotspot`, `unresolved`, `referral`, `none`), `csi_resolve` numeric, `would_recommend` bool, `unresolved_shop` bool, `referral_consumer` bool, `alert_posted_at` timestampz null, `referral_letter_posted_at` timestampz null; INDEX `idx_survey_match` (`repair_customer_id`, `submitted_at`)
- Verify: `supabase migration up` clean; insert/select round-trips each column
- Files: `supabase/migrations/<ts>_extend_survey_responses.sql`
- [] Task: Extend `repair_customers` with match + referral columns
- Acceptance: adds `legacy_match_key` text (indexed), `referral_tracking_enabled` bool, `credit_hold` bool
- Verify: `supabase migration up` clean; index exists on `legacy_match_key`
- Files: `supabase/migrations/<ts>_extend_repair_customers.sql`
- [] Task: Extend `company_programs` with `referral_tracking_enabled`
- Acceptance: adds `referral_tracking_enabled` bool; `customizations_jsonb` already holds logo/header/footer/greeting (no change)
- Verify: `supabase migration up` clean
- Files: `supabase/migrations/<ts>_extend_company_programs.sql`

- [] Task: Add ops-backbone table migrations (companies, employees, repair_orders, estimates, import_templates)
- Acceptance: companies, employees, repair_orders, estimates, import_templates created per PLANNING Data Model; repair_orders.ro_completed_at present for eligibility windowing; FKs wired
- Verify: supabase migration up clean; list_tables shows all five
- Files: supabase/migrations/<ts>_create_ops_backbone.sql
- [] Task: Add master-data table migrations (products, items, vehicles, insurance_companies, insurance_agents)
- Acceptance: five master-data tables per PLANNING SysConfig section
- Verify: supabase migration up clean
- Files: supabase/migrations/<ts>_create_master_data.sql
- [] Task: Add security_profiles + assignments migration and RLS gating
- Acceptance: security_profiles (id, name, is_builtin, functions_jsonb), user_security_profile_assignments; RLS on ops tables gated by functions_jsonb flags for psg_internal; psg_superadmin bypass
- Verify: supabase migration up clean; RLS policy present on companies
- Files: supabase/migrations/<ts>_create_security_profiles.sql, supabase/migrations/<ts>_ops_rls_policies.sql
- [] Task: Seed roles, modules, and built-in Administrator security profile
- Acceptance: seed inserts the Administrator profile granting all flags (manage_companies, manage_repair_customers, manage_surveys, manage_production, manage_reports, manage_sys_config, manage_data_imports, manage_users); ops modules registered
- Verify: node scripts/seed.mjs runs idempotently; query shows Administrator with all flags true
- Files: scripts/seed.mjs

Ops CRUD

- [] Task: Companies + employees + programs endpoints
- Acceptance: /api/ops/companies (list/create), /api/companies/[id] (get/update/delete), /api/companies/[id]/employees, /api/companies/[id]/programs; Zod validation; gated by manage_companies
- Verify: pnpm test covers handler happy + denied path; manual create persists with RLS
- Files: src/app/api/ops/companies/route.ts, src/app/api/ops/companies/[id]/route.ts, src/app/api/ops/companies/[id]/employees/route.ts, src/app/api/ops/companies/[id]/programs/route.ts
- [] Task: Repair customers + repair orders endpoints
- Acceptance: /api/ops/repair-customers (+ [id]), /api/ops/repair-orders (+ [id]); list/filter/sort/search; gated by manage_repair_customers
- Verify: pnpm test; manual add-new RO links to repair_customer
- Files: src/app/api/ops/repair-customers/route.ts, src/app/api/ops/repair-customers/[id]/route.ts, src/app/api/ops/repair-orders/route.ts, src/app/api/ops/repair-orders/[id]/route.ts
- [] Task: Estimates + surveys (entry/view) endpoints

- Acceptance: `/api/ops/estimates (+ [id] )`, `/api/ops/surveys (+ [id] )` for entry and view; survey entry writes the classification source columns; gated by `manage_surveys`
- Verify: `pnpm test`; survey entry persists `csi_resolve`, `would_recommend`, etc.
- Files: `src/app/api/ops/estimates/route.ts`, `src/app/api/ops/estimates/[id]/route.ts`, `src/app/api/ops/surveys/route.ts`, `src/app/api/ops/surveys/[id]/route.ts`
- [] Task: Master-data (sys-config) CRUD endpoints
- Acceptance: `/api/ops/sys-config/{products,items,vehicles,insurance-companies,insurance-agents}` CRUD each; gated by `manage_sys_config`
- Verify: `pnpm test` on one resource; manual CRUD on each
- Files: `src/app/api/ops/sys-config/products/route.ts`, `src/app/api/ops/sys-config/vehicles/route.ts`, `src/app/api/ops/sys-config/insurance-companies/route.ts`, `src/app/api/ops/sys-config/insurance-agents/route.ts`, `src/app/api/ops/sys-config/items/route.ts`
- [] Task: `import_templates` importer (absorbs psg-import, replaces 325 per-shop scripts)
- Acceptance: `/api/ops/import-templates (+ [id] )` for per-company `field_mapping_jsonb`; `/api/ops/repair-orders/import` and `/api/ops/estimates/import` accept upload (xlsx/xlsx/csv/txt), validate against template, load rows, surface row-level errors; gated by `manage_data_imports`
- Verify: `pnpm test` on mapping validation; manual RO import from sample xlsx loads and reports a deliberate bad row
- Files: `src/lib/import/templateImporter.ts`, `src/app/api/ops/import-templates/route.ts`, `src/app/api/ops/repair-orders/import/route.ts`, `src/app/api/ops/estimates/import/route.ts`
- [] Task: Ops UI route group skeleton
- Acceptance: `/ops/{companies,repair-customers,repair-orders,estimates,surveys,sys-config}` list/detail pages behind the ops shell; reuse the dashboard component pattern; desktop-first
- Verify: `turbo run build --filter=psg-hub`; pages render gated by security profile
- Files: `src/app/(ops)/ops/companies/page.tsx`, `src/app/(ops)/ops/repair-customers/page.tsx`, `src/app/(ops)/ops/repair-orders/page.tsx`, `src/app/(ops)/ops/surveys/page.tsx`, `src/app/(ops)/ops/layout.tsx`

Survey scoring + ingest

- [] Task: Survey scoring service (`classifySurvey`)
- Acceptance: pure function returning `alert_class` from `csi_resolve`, `would_recommend`, `unresolved_shop`, `referral_consumer`, plus per-company `referral_tracking_enabled` and `credit_hold`; encodes the v3 LIVE DDR rules exactly (perfect, misfire, hotspot, unresolved, referral, none)
- Verify: `pnpm test` with a case table covering all six classes incl. referral gate (`referral_tracking_enabled` true and `credit_hold` false) and misfire (satisfied, would-recommend, `csi_resolve < 1`)
- Files: `src/lib/surveys/scoring/classifySurvey.ts`, `src/lib/surveys/scoring/classifySurvey.test.ts`
- [] Task: Survey ingest cron worker

- Acceptance: `/api/cron/survey-ingest` pulls new `survey_responses` since watermark, resolves `repair_customer_id` (FK; fall back to `legacy_match_key` during migration), runs `classifySurvey`, stamps `alert_posted_at`; unmatched rows kept with null FK and queued; returns counts `{ingested, classified:{...} }`; cron-secret header auth
- Verify: `pnpm test` on the worker; seeded run classifies and stamps; re-run does not double-stamp; unmatched row appears in needs-match query
- Files: `src/app/api/cron/survey-ingest/route.ts`, `src/lib/surveys/ingest.ts`
- [] Task: Needs-match queue surface
- Acceptance: `/api/ops/surveys/needs-match` lists surveys with null `repair_customer_id`; `/ops/surveys/needs-match` page lets an account manager assign the FK; assignment re-runs classification for that row
- Verify: `pnpm test`; manual assign resolves a queued survey and clears it from the queue
- Files: `src/app/api/ops/surveys/needs-match/route.ts`, `src/app/(ops)/ops/surveys/needs-match/page.tsx`

Eligibility + suppression

- [] Task: Eligibility service with suppression rule
- Acceptance: computes per `letter_kind` over the RO completion-date window (3/6/12/18/24 month, birthday, drivers_license); excludes any customer with an open survey alert in the window and sets `suppressed_by_alert=true`; uses `ro_completed_at` not survey date; upserts idempotently on `(repair_customer_id, letter_kind, period_key)`
- Verify: `pnpm test` covering eligible, suppressed-by-open-alert, and no-duplicate-on-rerun
- Files: `src/lib/eligibility/compute.ts`, `src/lib/eligibility/windows.ts`, `src/lib/eligibility/compute.test.ts`
- [] Task: Eligibility-compute cron worker
- Acceptance: `/api/cron/eligibility-compute` accepts optional `{ letter_kind }` (all kinds if omitted), calls the eligibility service, returns `{ computed, eligible, suppressed }`; idempotent; cron-secret header auth; freshness alert if prior `computed_at` stale beyond 36h
- Verify: `pnpm test`; two consecutive runs produce no duplicate `letter_eligibility` rows
- Files: `src/app/api/cron/eligibility-compute/route.ts`
- [] Task: v1.1 verification — Playwright happy path
- Acceptance: E2E creates a company, adds employees, imports an RO from a sample file, enters a survey, confirms classification, runs eligibility, and asserts a `suppressed_by_alert` row when an open alert exists
- Verify: `pnpm --filter=psg-hub exec playwright test ops-foundation`
- Files: `tests/e2e/ops-foundation.spec.ts`

Milestone v1.3 — Production + Cutover

- [] Task: Mail adapter interface + Lob and in-house adapters
- Acceptance: shared `MailAdapter` interface; `LobAdapter` (Lob API + address verification) and `InHouseAdapter` (PDF generate to print handoff); per-template/per-shop vendor selection; writes `mail_vendor_jobs` with vendor enum (lob, inhouse)
- Verify: `pnpm test` mocks both adapters; selection logic picks vendor per template

- Files: `src/lib/production/mail/MailAdapter.ts`, `src/lib/production/mail/LobAdapter.ts`, `src/lib/production/mail/InHouseAdapter.ts`, `src/lib/production/mail/selectVendor.ts`
 - [] Task: Production batch build service + endpoint
 - Acceptance: `/api/ops/production/batches` (list/create) builds from eligible customers given `{ name (unique), product_ids[], company_ids[]|null }`; each `production_document` gets a unique `print_id`; returns `{ batch_id, document_count }`; gated by `manage_production`
 - Verify: `pnpm test`; build over seeded eligibility yields documents with distinct `print_id`
 - Files: `src/lib/production/buildBatch.ts`, `src/app/api/ops/production/batches/route.ts`
 - [] Task: Idempotent print transition endpoint
 - Acceptance: `/api/ops/production/batches/[id]/print` sets `production_documents.status=printed`, `printed_at=now()`, `printed_by_profile_id` only where `status=unprinted`; returns `{ printed, skipped_already_printed }`
 - Verify: `pnpm test` asserts second print run reports `skipped_already_printed` equal to first run's `printed` and changes nothing
 - Files: `src/lib/production/printBatch.ts`, `src/app/api/ops/production/batches/[id]/print/route.ts`
 - [] Task: Reprint with audit + historical search
 - Acceptance: `/api/ops/production/documents/[id]/reprint` writes `production_reprint_log` (`document_id`, `reprinted_by_profile_id`, `reprinted_at`); `/api/ops/production/historical` searches by batch name, `print_id`, company, product, repair customer
 - Verify: `pnpm test`; reprint creates a log row; historical search returns by `print_id`
 - Files: `src/app/api/ops/production/documents/[id]/reprint/route.ts`, `src/app/api/ops/production/historical/route.ts`
 - [] Task: Lob status webhook
 - Acceptance: `/api/webhooks/lob` verifies HMAC signature, is idempotent on event id, updates the matching `mail_vendor_jobs.status`; returns 200
 - Verify: `pnpm test` with a signed fixture event updates status; replayed event is a no-op
 - Files: `src/app/api/webhooks/lob/route.ts`, `src/lib/production/mail/lobWebhook.ts`
 - [] Task: Production UI (Outlook-inspired layout) + cutover
 - Acceptance: `/ops/production` (batch list left, document list right), `/ops/production/new` (batch wizard), `/ops/production/historical`; explicit confirm + dry-run before print (irreversible mail spend); cutover points production team at psg-hub, FileMaker read-only
 - Verify: `turbo run build --filter=psg-hub`; manual batch-to-print flow with confirm gate
 - Files: `src/app/(ops)/ops/production/page.tsx`, `src/app/(ops)/ops/production/new/page.tsx`, `src/app/(ops)/ops/production/historical/page.tsx`
 - [] Task: v1.3 verification — Playwright happy path
 - Acceptance: E2E builds a batch from an eligible customer, prints it (only unprinted transition), moves to historical, reprints with audit, and asserts a Lob webhook updates `mail_vendor_jobs`
 - Verify: `pnpm --filter=psg-hub exec playwright test production`
 - Files: `tests/e2e/production.spec.ts`
-

Milestone v1.4 — Operational Reports

- [] Task: Report engine framework
 - Acceptance: `/api/ops/reports/[reportSlug]` parameter-driven runner (date range + filters); uses `pg` pool plus Redis cache, not PostREST; supports CSV/Excel/PDF export; gated by `manage_reports`
 - Verify: `pnpm test` on the runner with a stub query; cache hit on repeat call
 - Files: `src/lib/reports/engine.ts`, `src/lib/reports/cache.ts`, `src/app/api/ops/reports/[reportSlug]/route.ts`, `src/app/(ops)/ops/reports/page.tsx`
 - [] Task: First report end to end — Perfect Score
 - Acceptance: `perfect-score` report queries surveys with `alert_class='perfect'` over the date range; rows match a hand-checked query; renders in the report index and exports
 - Verify: `pnpm test` asserts row set equals expected over seeded data; LCP < 3s on 12-month range
 - Files: `src/lib/reports/reports/perfectScore.ts`, `src/app/(ops)/ops/reports/[reportSlug]/page.tsx`
 - [] Task: Survey-alert reports batch (mis-fire, hot-spot, unresolved-issue, referral-noted, referral-comparison)
 - Acceptance: each slug queries by its `alert_class`; `referral-comparison` compares referral volume across periods; all served by the engine
 - Verify: `pnpm test` per slug over seeded data
 - Files: `src/lib/reports/reports/misFire.ts`, `src/lib/reports/reports/hotSpot.ts`, `src/lib/reports/reports/unresolvedIssue.ts`, `src/lib/reports/reports/referralNoted.ts`, `src/lib/reports/reports/referralComparison.ts`
 - [] Task: Volume + invoicing + CSI report batches (remaining 20 slugs)
 - Acceptance: the remaining PLANNING report slugs (processing-recap, invoicing-recap, reprint-recap, pay-type-analysis, vehicle-analysis-make/model, referral-directory, recap-trailing, agent-capture, agent-sales, claims-review, audit, name-recap-by-shop, monthly-csi-display, performance-dashboard, market-dashboard, estimator-csi, body-tech-performance, painter-performance, survey-alert-recap, rental-car-analysis) each resolve through the engine
 - Verify: `pnpm test` smoke per slug; all 26 slugs resolve from the index
 - Files: `src/lib/reports/reports/volumeInvoicing.ts`, `src/lib/reports/reports/csi.ts`, `src/lib/reports/reports/customerInsurance.ts`, `src/lib/reports/registry.ts`
 - [] Task: v1.4 verification — Playwright happy path
 - Acceptance: E2E opens the report index, runs the Perfect Score report with a date range, and exports it
 - Verify: `pnpm --filter=psg-hub exec playwright test reports`
 - Files: `tests/e2e/reports.spec.ts`
-

Milestone v1.3.5 — Migration (optional)

Built only if the optional add-on is triggered (PLANNING Q15, SDD ADR-5). Functional replacement (v1.1 to v1.4) does not depend on it. Files are under `psg-data-lake/`, not `apps/psg-hub/`.

- [] Task: Migration extract readers (DDR CSV + FileMaker Data API)
- Acceptance: `fm_extract` reads DDR CSV exports and the FileMaker Data API; tolerates absent stale references (BridgeSystem, EmployeeSatisfaction, Import Flush2, PSG_SurveyExport_07.1); never sources from a workstation path
- Verify: `python -m pytest psg-data-lake/tests/test_fm_migrate.py -k extract`
- Files: `psg-data-lake/fm_extract/csv_reader.py`, `psg-data-lake/fm_extract/data_api.py`
- [] Task: Match-key resolution + PII redaction + dedupe transform
- Acceptance: `fm_transform` resolves `legacy_match_key` to `repair_customer_id` FKs, redacts PII per `psg_sensitive_pii_*` patterns, dedupes
- Verify: `python -m pytest psg-data-lake/tests/test_fm_migrate.py -k transform`
- Files: `psg-data-lake/fm_transform/resolve_keys.py`, `psg-data-lake/fm_transform/redact.py`
- [] Task: Migration orchestrator + reconciliation
- Acceptance: `fm_migrate.py` runs extract to transform to load in the sequenced order (master data, companies/employees, repair_customers, ROs/estimates, survey_responses, production history); reconciles counts to the DDR before each next step
- Verify: `python psg-data-lake/fm_migrate.py --entity repair_customers --dry-run`; counts reconcile (repair_customers approx 281K, survey_responses approx 334K); 50-link spot-check resolves; unmatched rate under 2 percent
- Files: `psg-data-lake/fm_migrate.py`, `psg-data-lake/tests/test_fm_migrate.py`

Schema Migration Spec

This spec defines the concrete, forward-only Postgres DDL that implements the **Data Storage Changes** in the SDD (`../solution-design.md` , section "Interface Specifications → Data Storage Changes") and the five Architecture Decisions (ADR-1 through ADR-5, see `../adr/`).

The deltas re-platform FileMaker Advantage behavior onto the **already-planned** psg-hub schema (`projects/psg-hub/PLANNING.md` , Data Model). Most target tables already exist. This spec adds only the behavioral columns the DDR analysis proved are required, the survey↔customer foreign key (ADR-2), the explicit suppression flag (ADR-4), and one new table (`letter_eligibility`).

Scope

- New enum `alert_class` : the survey alert classifications.
- New enum `letter_kind` : the letter eligibility cycle buckets.
- `ALTER TABLE survey_responses` : alert columns, the real FK, idempotency stamps.
- `ALTER TABLE repair_customers` : legacy reconciliation key + referral/credit gates.
- `ALTER TABLE company_programs` : per-shop referral tracking gate.
- `CREATE TABLE letter_eligibility` : eligibility + suppression + print idempotency.
- RLS notes per table.
- Migration file naming convention + forward-only policy.

Reused as-is (no DDL here)

`production_batches` , `production_documents` , `production_reprint_log` , `mail_vendor_jobs` , `email_jobs` , `sms_jobs` , `import_templates` , and `security_profiles` are already defined in `PLANNING.md` (Data Model → Production module, Ops backbone, Security profiles). They are reused as-is. This spec does not redefine them.

Conventions inherited from psg-hub

These are not invented here. They come from the shipped migrations (`apps/psg-hub/supabase/migrations/`) and `PLANNING.md` :

- **profile_id everywhere** (never `user_id`) for actor references; foreign-keyed to `public.profiles(id)` .
 - **Shop clamp**: customer/PII rows are clamped to authorized shops with `shop_id IN (select public.user_shop_ids())` .
 - **Ops capability gate**: ops tables are gated by `private.current_user_has_fn('<function>')` , which returns true for `psg_superuser` , or for `psg_internal` when `security_profiles.functions_jsonb` carries the function key. This is the `security_profiles.functions_jsonb` gate the SDD requires.
 - **Default-deny**: every new table is `enable row level security` with no blanket `anon` / `authenticated` policy. Service-role (cron + Python worker) bypasses RLS; interactive access goes through the named policies below.
 - **Idempotent DDL**: `create ... if not exists` , `add column if not exists` , `do $$ ... $$` enum guards, so a re-run is a no-op (forward-only, run-once safe).
-

Migration files

Forward-only, timestamped, one concern per file, landing under `apps/psg-hub/supabase/migrations/`. Naming follows the existing convention (`YYYYMMDDHHMMSS_snake_case_description.sql`). Proposed files for this spec:

```
apps/psg-hub/supabase/migrations/  
20260609090000_filemaker_enums.sql          # alert_class, letter_kind  
20260609090100_survey_responses_alerts.sql   # FK + alert columns + stamps + index  
20260609090200_repair_customers_referral.sql # legacy_match_key + referral/credit gates  
20260609090300_company_programs_referral.sql # referral_tracking_enabled  
20260609090400_letter_eligibility.sql        # new table + RLS + indexes
```

Apply with `supabase migration up`. Production apply goes through the operator gate (advisor baseline + diff) per the project migration-safety protocol; build applies local-first.

1. Enums

`alert_class` encodes the five DDR survey classifications plus `none`. `letter_kind` encodes the eligibility cycle buckets the Advantage file drove off RO completion date plus the event letters.

```
-- 20260609090000_filemaker_enums.sql  
-- Forward-only. Enum creation guarded so a re-run is a no-op.  
  
do $$  
begin  
  if not exists (select 1 from pg_type where typename = 'alert_class') then  
    create type public.alert_class as enum (  
      'perfect',      -- SQ_Alert_Perfect_post  
      'misfire',      -- satisfied + would-recommend + not perfect (missed referral)  
      'hotspot',      -- negative result needing follow-up  
      'unresolved',   -- SQ_Unresolved_Shop  
      'referral',     -- SQ_Alert_Referral_post  
      'none'          -- no alert  
    );  
  end if;  
end$$;  
  
do $$  
begin  
  if not exists (select 1 from pg_type where typename = 'letter_kind') then  
    create type public.letter_kind as enum (  
      'three_month',  -- SS - Eligibility - 3Month  
      'six_month',  
      'one_year',  
      'eighteen_month',  
      'two_year',  
      'birthday',  
      'drivers_license',  
      'thank_you',  
      'referral'  
    );  
  end if;  
end$$;
```

To add an alert class or letter kind later, ship a forward-only `alter type ... add value if not exists` migration. Do not edit the enum in place.

2. `survey_responses` (EXTEND)

Adds the real foreign key (ADR-2) that replaces the composite shop-id+month+year string match key, the alert columns from the DDR classification calcs, and the idempotency stamps that replace the FileMaker `SQ_Alert_PostDate_*` and `S_RC_Referral_Letter_Post` date stamps. The FK is nullable until matched; an unmatched survey is retained and surfaced in the needs-match queue (never dropped).

```
-- 20260609090100_survey_responses_alerts.sql
-- Forward-only. survey_responses already exists (PLANNING Data Model, v0.3).

alter table public.survey_responses
  add column if not exists repair_customer_id uuid
    references public.repair_customers(id) on delete set null, -- ADR-2: real FK, nullable until
  matched
  add column if not exists alert_class public.alert_class not null default 'none',
  add column if not exists csi_resolve numeric, -- SQ_Scale_CSI_Resolve
  add column if not exists would_recommend boolean, -- SQ_Would_Recom_Facility
  add column if not exists unresolved_shop boolean, -- SQ_Unresolved_Shop
  add column if not exists referral_consumer boolean, -- SQ_Referral_Consumer
  add column if not exists alert_posted_at timestampz, -- replaces SQ_Alert_PostDate_*
  stamps
  add column if not exists referral_letter_posted_at timestampz; -- replaces
  S_RC_Referral_Letter_Post

-- Survey->customer match lookup + suppression-window scan (alert_posted_at).
create index if not exists idx_survey_match
  on public.survey_responses (repair_customer_id, submitted_at);
```

RLS: `survey_responses` holds end-consumer PII and is shop-scoped. Clamp to authorized shops; ops mutation gated by `manage_surveys`. Service-role (survey-ingest cron) bypasses RLS.

```
alter table public.survey_responses enable row level security;

drop policy if exists survey_responses_select on public.survey_responses;
create policy survey_responses_select on public.survey_responses
  for select to authenticated
  using (
    shop_id in (select public.user_shop_ids())
    or private.current_user_has_fn('manage_surveys')
  );

drop policy if exists survey_responses_write on public.survey_responses;
create policy survey_responses_write on public.survey_responses
  for all to authenticated
  using (private.current_user_has_fn('manage_surveys'))
  with check (private.current_user_has_fn('manage_surveys'));
```

3. `repair_customers` (EXTEND)

`legacy_match_key` keeps the old `RC_MatchField_Master` string for migration reconciliation only (ADR-2 fallback during the v1.3.5 load). The referral/credit gates drive the Referral vs Misfire branch in `classifySurvey` (SDD Implementation Examples).

```
-- 20260609090200_repair_customers_referral.sql
-- Forward-only. repair_customers already exists (PLANNING Data Model, v1.1).
```

```

alter table public.repair_customers
  add column if not exists legacy_match_key text,          --
RC_MatchField_Master (reconciliation only)
  add column if not exists referral_tracking_enabled boolean not null default false, -- Master
Client::M_ReferralNoted_flag
  add column if not exists credit_hold boolean not null default false;      -- M_CreditHold_flag
(suppresses referral path)

-- Migration fallback match: resolve legacy keys to FKs during v1.3.5 ETL.
create index if not exists idx_repair_customers_legacy_match_key
  on public.repair_customers (legacy_match_key);

```

RLS: `repair_customers` is high-sensitivity end-consumer PII. Clamp to authorized shops; ops mutation gated by `manage_companies`. Follows the existing `psg_sensitive_pii_*` redaction patterns at the column/view layer.

```

alter table public.repair_customers enable row level security;

drop policy if exists repair_customers_select on public.repair_customers;
create policy repair_customers_select on public.repair_customers
  for select to authenticated
  using (
    shop_id in (select public.user_shop_ids())
    or private.current_user_has_fn('manage_companies')
  );

drop policy if exists repair_customers_write on public.repair_customers;
create policy repair_customers_write on public.repair_customers
  for all to authenticated
  using (private.current_user_has_fn('manage_companies'))
  with check (private.current_user_has_fn('manage_companies'));

```

Note: if `repair_customers` carries `company_id` rather than `shop_id`, swap the clamp to the company→shop join your shop-membership helper exposes. The gate (`manage_companies`) is unchanged.

4. `company_programs` (EXTEND)

Per-shop referral tracking gate. Read cross-file in FileMaker; seed it here before classification runs, or every survey misclassifies (SDD Implementation Gotchas). `customizations_jsonb` already holds logo/header/footer/greeting and is not changed here.

```

-- 20260609090300_company_programs_referral.sql
-- Forward-only. company_programs already exists (PLANNING Data Model, v1.1).

alter table public.company_programs
  add column if not exists referral_tracking_enabled boolean not null default false; -- drives
Referral vs Misfire per shop

```

RLS: ops master-data table, gated by `manage_companies`.

```

alter table public.company_programs enable row level security;

drop policy if exists company_programs_rw on public.company_programs;
create policy company_programs_rw on public.company_programs
  for all to authenticated

```

```
using (private.current_user_has_fn('manage_companies'))
with check (private.current_user_has_fn('manage_companies'));
```

5. letter_eligibility (NEW)

The eligibility + suppression + print-idempotency table. Replaces the `SS - Eligibility - *` date-window finds, makes the suppression rule explicit (`suppressed_by_alert`, ADR-4), and replaces the `Letter_*_Printed` date stamps with `printed_at`. The `UNIQUE (repair_customer_id, letter_kind, period_key)` constraint is the no-double-send guard (ADR-3 idempotency).

`period_key` is the cycle bucket (for example `date_trunc('month', ro_completed_at)` rendered as text) so re-running eligibility for the same customer + letter + cycle is a no-op via `on conflict do nothing / do update`.

```
-- 20260609090400_letter_eligibility.sql
-- Forward-only. New table. Idempotent create.

create table if not exists public.letter_eligibility (
  id uuid primary key default gen_random_uuid(),
  repair_customer_id uuid not null
    references public.repair_customers(id) on delete cascade,
  letter_kind public.letter_kind not null,
  eligible boolean not null default false,
  suppressed_by_alert boolean not null default false, -- ADR-4: the survey-alert suppression
  rule, explicit
  period_key text not null, -- cycle bucket (e.g. month of
  ro_completed_at)
  computed_at timestamptz not null default now(),
  printed_at timestamptz, -- replaces Letter_*_Printed stamps
  (idempotency)
  constraint letter_eligibility_unique_cycle
    unique (repair_customer_id, letter_kind, period_key) -- no double-send
);

-- Cron upsert target + "what is eligible right now" reads.
create index if not exists idx_letter_eligibility_customer_kind
  on public.letter_eligibility (repair_customer_id, letter_kind);

-- Batch builder: pull eligible, not-yet-printed rows for a cycle.
create index if not exists idx_letter_eligibility_ready
  on public.letter_eligibility (letter_kind, period_key)
  where eligible and printed_at is null;
```

RLS: ops table tied to end-consumer records. Gated by the production capability (`manage_production`, since this feeds the print queue). Service-role (eligibility-compute cron) bypasses RLS for the idempotent upsert.

```
alter table public.letter_eligibility enable row level security;

drop policy if exists letter_eligibility_select on public.letter_eligibility;
create policy letter_eligibility_select on public.letter_eligibility
  for select to authenticated
  using (private.current_user_has_fn('manage_production'));

drop policy if exists letter_eligibility_write on public.letter_eligibility;
create policy letter_eligibility_write on public.letter_eligibility
  for all to authenticated
  using (private.current_user_has_fn('manage_production'))
  with check (private.current_user_has_fn('manage_production'));
```

RLS summary

Table	Clamp	Ops gate (<code>current_user_has_fn</code>)	Notes
<code>survey_responses</code>	<code>shop_id IN user_shop_ids()</code>	<code>manage_surveys</code>	PII; survey-ingest cron is service-role
<code>repair_customers</code>	<code>shop_id IN user_shop_ids()</code>	<code>manage_companies</code>	high-sensitivity PII; <code>psg_sensitive_pii_*</code> redaction at view layer
<code>company_programs</code>	n/a (master data)	<code>manage_companies</code>	seed referral flag before classification
<code>letter_eligibility</code>	via <code>manage_production</code>	<code>manage_production</code>	eligibility-compute cron is service-role

`psg_superuser` bypasses the function gate (handled inside `private.current_user_has_fn`). All four tables are default-deny: no blanket `anon` / `authenticated` policy exists beyond the named policies above.

Forward-only policy

- Migrations are **forward-only**. No down/rollback scripts. New columns and tables are additive, so a UI rollback leaves data intact, and FileMaker stays authoritative until the v1.3 cutover (rollback = point users back to FileMaker).
- All DDL is idempotent (`if not exists` , enum guards, `drop policy if exists` before `create policy`) so a re-apply is a no-op.
- Enum changes go through `alter type ... add value if not exists` ; never edit an enum in place.
- Schema migrations must not collide with the BSM, advantage-portal, ads-dashboard, and local_reach schemas in the shared Supabase project `gylkkzmcmbdftxieyabw` . Resolve against the latest applied migration before timestamping a new file.

Test Plan

This test plan turns the SDD (`solution-design.md`) into a concrete, executable test strategy for the FileMaker Advantage integration. It mirrors the SDD Acceptance Criteria and Quality Requirements, expands them into runnable test cases, and aligns to the psg-hub milestone plan (`PLANNING.md`). The DDR analysis remains the behavioral source of truth; every classification, suppression, idempotency, and migration case below derives from it.

The plan is scoped to the ops backbone the SDD delivers across v1.1 → v1.4, the optional migration add-on v1.3.5, and the PII sign-off gates at v1.1, v1.3, and v2.0.

1. Test Strategy

1.1 Levels

Level	Tool	What it covers	Where it lives
Unit	Vitest 4 + jsdom	Pure logic and services in <code>src/lib/</code> (eligibility windows, suppression rule, survey scoring, production idempotency). No network, no real DB.	<code>apps/psg-hub/src/lib/**/*test.ts</code>
Component	Vitest 4 + jsdom + Testing Library	Ops UI components (needs-match queue, batch print confirm, report parameter forms). Rendered, no real backend.	<code>apps/psg-hub/src/app/(ops)/**/*test.tsx</code>
Integration	Vitest 4 against a test Supabase project	API route handlers (<code>/api/ops/*</code> , <code>/api/cron/*</code> , <code>/api/webhooks/lob</code>) against a real Postgres schema with the SDD migrations applied and seeded fixtures. Verifies SQL, unique constraints, RLS, and upsert idempotency.	<code>apps/psg-hub/src/app/api/**/*integration.test.ts</code>
E2E	Playwright	One ops happy path per milestone, browser-driven against a seeded staging stack.	<code>apps/psg-hub/e2e/**/*spec.ts</code>
Migration	Vitest + Python <code>pytest</code>	Reconciliation and spot-check assertions on the v1.3.5 ETL worker output.	<code>psg-data-lake/tests/test_fm_migrate.py</code> and <code>apps/psg-hub/src/lib/migration/*test.ts</code>

1.2 Approach by subsystem

- Survey scoring and eligibility are pure functions with high behavioral risk. Cover them exhaustively at the unit level first, then prove the SQL path at the integration level.
- API routes get integration tests against a test Supabase project (a dedicated branch of `gylkkzmcmbdftxiyabw` , never production). Each cron and ops route runs against applied migrations and seed fixtures.
- The mail vendor webhook and external calls (Lob, SendGrid, Twilio, FileMaker Data API) are mocked at the unit and integration levels. Lob test mode drives the v1.3 E2E happy path.
- E2E proves the wired happy path only; it is not the place for branch-coverage of classification or eligibility.

1.3 Commands

```
pnpm test # Vitest 4 (unit + component + integration)
pnpm --filter=psg-hub exec playwright test # E2E
```

```
pnpm lint && pnpm typecheck # static gates, run before tests in CI
python -m pytest psg-data-lake/tests/ # migration worker (v1.3.5)
```

2. Coverage Expectations and Quality Bars

Coverage and quality thresholds come straight from the SDD Quality Requirements and PLANNING.md Quality Gates. They are gates, not targets.

Bar	Threshold	Source	Verified by
Vitest line coverage (new code)	≥ 70% lines	PLANNING Quality Gates	CI coverage report on changed files
<code>classifySurvey</code> branch coverage	100% of the six alert classes	SDD <code>classifySurvey</code> example	Unit test matrix (§4.1)
Eligibility + suppression branch coverage	in-window and out-of-window, with and without open alert	SDD suppression SQL	Unit + integration (§4.2)
LCP on <code>/ops/*</code> and <code>/ops/reports/*</code>	< 3s	SDD Performance; PLANNING	Playwright trace + Lighthouse in E2E
Cron completion	eligibility-compute and survey-ingest each < 5 min over full population	SDD Performance	Integration timing assertion on full-size fixture
Idempotency	zero double-sends; re-running any cron or print produces no new rows or sends	SDD Reliability	Integration re-run assertions (§4.3, §4.4)
Unmatched-survey rate	visible and < 2% after migration reconciliation	SDD Reliability + Migration integrity	Migration reconciliation (§5)
Migration count reconciliation	<code>repair_customers</code> ≈ 281K, <code>survey_responses</code> ≈ 334K within tolerance	SDD Migration integrity	Reconciliation checks (§5)
Customer↔survey link spot check	50 links resolve correctly	SDD Migration integrity	Spot-check fixture (§5)
Webhook idempotency	every webhook handler keyed; replayed events are no-ops	PLANNING Webhook idempotency	Integration replay test (§4.5)
Import validation	every import validates against its <code>import_templates</code> mapping	PLANNING Import validation	Integration test on import endpoint
PII review	manual sign-off	SDD Security; PLANNING	Sign-off checklist (§6)

Tolerance for migration counts: ± 0.5% per entity, with every excluded row logged and attributed to a known DDR data issue (stale external references per SDD Risks). Counts outside tolerance block cutover.

3. Acceptance Criteria (EARS), by subsystem

Each criterion below is an EARS statement traceable to the SDD Acceptance Criteria, expanded into the concrete test cases that prove it. AC IDs are stable references for the test suite.

3.1 Surveys + alerts (v1.1 / v1.4)

AC	EARS statement	Test cases
AC-S1	WHEN a survey is ingested, THE SYSTEM SHALL resolve its <code>repair_customer_id</code> and set <code>alert_class</code> using the DDR classification rules.	UT-CLS-1..6 (§4.1); IT-INGEST-1 (resolves FK then classifies); IT-INGEST-2 (falls back to <code>legacy_match_key</code> during migration)
AC-S2	IF a survey cannot be matched to a repair customer, THEN THE SYSTEM SHALL retain it and surface it in a needs-match queue.	UT-MATCH-1 (§4.5); IT-INGEST-3 (row kept, FK null, appears in needs-match queue, not dropped)
AC-S3	WHILE a company has <code>referral_tracking_enabled</code> false, THE SYSTEM SHALL classify an otherwise-referral survey as <code>misfire</code> , not <code>referral</code> .	UT-CLS-2 and UT-CLS-5 (§4.1)
AC-S4	IF a repair customer is on <code>credit_hold</code> , THEN THE SYSTEM SHALL NOT classify the survey as <code>referral</code> .	UT-CLS-5b (§4.1)

3.2 Eligibility + suppression (v1.3)

AC	EARS statement	Test cases
AC-E1	WHEN eligibility is computed, THE SYSTEM SHALL exclude any customer with an open survey alert in the window and record <code>suppressed_by_alert</code> .	UT-SUP-1, UT-SUP-2 (§4.2); IT-ELIG-1 (suppressed row has <code>eligible=false</code> , <code>suppressed_by_alert=true</code>)
AC-E2	THE SYSTEM SHALL never create a duplicate letter for the same customer, <code>letter_kind</code> , and period.	UT-ELIG-IDEM-1 (§4.2); IT-ELIG-2 (re-run yields no new <code>letter_eligibility</code> row; ON CONFLICT DO NOTHING holds)
AC-E3	WHEN eligibility is computed, THE SYSTEM SHALL use RO completion date, not survey date, for the window.	IT-ELIG-3 (customer with old survey but in-window <code>ro_completed_at</code> is eligible; SDD gotcha)

3.3 Production (v1.3)

AC	EARS statement	Test cases
AC-P1	WHEN a batch is printed, THE SYSTEM SHALL transition only <code>unprinted</code> documents to <code>printed</code> and stamp <code>printed_at</code> .	UT-PRINT-1 (§4.3); IT-PRINT-1
AC-P2	WHEN a batch print is re-run, THE SYSTEM SHALL skip documents already <code>printed</code> and return <code>skipped_already_printed</code> .	UT-PRINT-2 (§4.3); IT-PRINT-2 (second run prints 0, skips all)
AC-P3	WHEN Lob returns a status, THE SYSTEM SHALL update the matching <code>mail_vendor_jobs</code> row.	IT-LOB-1 (HMAC-verified event updates status); IT-LOB-2 (bad signature rejected, no update)
AC-P4	WHEN a Lob webhook event is replayed, THE SYSTEM SHALL treat it as a no-op.	IT-LOB-3 (§4.5)

3.4 Reports (v1.4)

AC	EARS statement	Test cases
AC-R1	THE SYSTEM SHALL serve all 26 reports from a parameter-driven engine, including Perfect Score, Mis-Fire, Hot Spot, Unresolved Issue, and Referral Noted.	IT-RPT-1 (each of 26 slugs returns rows + meta for a seeded range); IT-RPT-2 (the five survey-response reports filter to their <code>alert_class</code>)
AC-R2	WHEN a report runs over a 12-month range, THE SYSTEM SHALL return within the LCP and report-cache budget.	IT-RPT-3 (Redis cache hit on second run); E2E-RPT timing

3.5 Migration (v1.3.5, optional)

AC	EARS statement	Test cases
AC-M1	WHERE the migration add-on is enabled, THE SYSTEM SHALL load historical entities and reconcile counts to the DDR before cutover.	MT-COUNT-1..6 (§5)
AC-M2	WHERE legacy keys are present, THE SYSTEM SHALL resolve <code>legacy_match_key</code> to a real <code>repair_customer_id</code> and retain the legacy key for reconciliation.	MT-LINK-1 (50-link spot check, §5)
AC-M3	IF a survey row cannot resolve to a repair customer during migration, THEN THE SYSTEM SHALL load it with a null FK into the needs-match queue and count it toward the unmatched rate.	MT-UNMATCH-1 (unmatched rate < 2%, §5)

4. Unit Test Cases

All inputs below name the exact SDD fields. The `classifySurvey` cases use the precise gate logic from the SDD example: `wouldRecommend`, `unresolvedShop`, `csiResolve`, `referralConsumer`, `referralTrackingEnabled` (on the company), and `creditHold`.

4.1 Survey alert classification (`src/lib/surveys/scoring`)

Gate order from the SDD `classifySurvey` :

1. `happy = wouldRecommend && !unresolvedShop`
2. `happy && referralConsumer && company.referralTrackingEnabled && !creditHold → referral`
3. `happy && csiResolve === 1 → perfect`
4. `happy && csiResolve < 1 → misfire`
5. `unresolvedShop → unresolved`
6. `csiResolve < 1 → hotspot`
7. `else → none`

One case per alert class, each chosen so exactly one gate fires. Columns name the SDD fields directly; `referralTrackingEnabled` and `creditHold` live on the company/customer the survey resolves to.

ID	Class	wouldRecommend	unresolvedShop	csiResolve	referralConsumer	referralTrackingEnabled	creditHold
UT-CLS-1	perfect	true	false	1	false	true	false
UT-CLS-2	misfire	true	false	0.8	false	true	false
UT-CLS-3	hotspot	false	false	0.6	false	true	false
UT-CLS-4	unresolved	true	true	0.5	false	true	false

ID	Class	wouldRecommend	unresolvedShop	csiResolve	referralConsumer	referralTrackingEnabled	creditHold
UT-CLS-5	referral	true	false	1	true	true	false
UT-CLS-6	none	false	false	1	false	true	false

Two branch cases prove the company/customer gates on the otherwise-referral input from UT-CLS-5:

ID	Branch	wouldRecommend	unresolvedShop	csiResolve	referralConsumer	referralTrackingEnabled	creditHold
UT-CLS-5b	tracking off blocks referral	true	false	1	true	false	false
UT-CLS-5c	credit hold blocks referral	true	false	1	true	true	true

Run the matrix as a single parameterized Vitest suite so any future change to gate order fails loudly.

4.2 Eligibility suppression and idempotency (`src/lib/eligibility`)

ID	Scenario	Setup	Expected
UT-SUP-1	In-window, open alert → suppressed	<code>ro_completed_at</code> in the three_month window; one <code>survey_responses</code> row with <code>alert_class <> 'none'</code> and <code>alert_posted_at</code> within 120 days	<code>eligible = false</code> , <code>suppressed_by_alert = true</code>
UT-SUP-2	In-window, no alert → eligible	same window; no open alert row	<code>eligible = true</code> , <code>suppressed_by_alert = false</code>
UT-SUP-3	In-window, only <code>none</code> alert → eligible	window; survey row with <code>alert_class = 'none'</code>	<code>eligible = true</code> (the SDD join excludes <code>alert_class = 'none'</code>)
UT-ELIG-IDEM-1	Re-compute, no duplicate	run <code>compute</code> twice for the same customer, <code>letter_kind = three_month</code> , same <code>period_key</code>	second run creates no new <code>letter_eligibility</code> row (UNIQUE on <code>repair_customer_id</code> , <code>letter_kind</code> , <code>period_key</code>)

4.3 Idempotent print (`src/lib/production`)

ID	Scenario	Setup	Expected
UT-PRINT-1	First print	batch with 3 <code>unprinted</code> documents	all 3 → <code>printed</code> , each <code>printed_at</code> stamped; returns <code>{ printed: 3, skipped_already_printed: 0 }</code>
UT-PRINT-2	Re-run skips printed	re-run print on the same batch	

ID	Scenario	Setup	Expected
			0 transitioned; returns { printed: 0, skipped_already_printed: 3 }; printed_at values unchanged
UT-PRINT-3	Mixed state	batch with 2 printed, 1 unprinted	returns { printed: 1, skipped_already_printed: 2 }

4.4 Eligibility idempotency at the service boundary

ID	Scenario	Setup	Expected
UT-ELIG-IDEM-2	Same customer, two cron runs	two eligibility-compute invocations, same customer + letter_kind + period_key	exactly one letter_eligibility row exists after both runs
UT-ELIG-IDEM-3	New period creates new row	second run with a different period_key (new cycle bucket)	a second row is created; the first is untouched

4.5 Unmatched survey and webhook replay

ID	Scenario	Setup	Expected
UT-MATCH-1	Unmatched survey lands in queue	ingest a survey whose repair_customer_id cannot be resolved by FK or legacy_match_key	row retained, FK null, flagged for needs-match; never dropped
UT-MATCH-2	Later match clears the queue	a previously unmatched survey gets a resolvable customer on re-ingest	FK populated, alert_class computed, removed from needs-match
UT-LOB-REPLAY	Webhook replay is a no-op	apply the same Lob event twice (same event id)	first updates mail_vendor_jobs.status; second changes nothing

5. Migration Test and Reconciliation Plan (v1.3.5)

The migration worker (psg-data-lake/fm_migrate.py) is tested in two layers: pytest on the transform/load logic, and Vitest integration assertions on the loaded Supabase state. Reconciliation runs per entity in the SDD sequencing order: master data → companies/employees → repair_customers → ROs/estimates → survey_responses → production history. Each step must pass before the next runs.

5.1 Count reconciliation

ID	Entity	Check	Pass condition
MT-COUNT-1	repair_customers	loaded row count vs DDR	≈ 281K within ± 0.5%; every excluded row logged with reason
MT-COUNT-2	survey_responses	loaded row count vs DDR	≈ 334K within ± 0.5%
MT-COUNT-3	companies / employees	loaded vs DDR master data	exact match (small reference sets)
MT-COUNT-4	repair_orders / estimates	loaded vs DDR	within ± 0.5%
MT-COUNT-5	production history	loaded vs DDR	within ± 0.5%

ID	Entity	Check	Pass condition
MT-COUNT-6	excluded-row ledger	every skipped row attributed	100% of exclusions map to a known DDR data issue (stale external references per SDD Risks)

5.2 Link spot check

ID	Check	Method	Pass condition
MT-LINK-1	50 customer↔survey links resolve correctly	sample 50 surveys with non-null <code>legacy_match_key</code> ; assert each resolved <code>repair_customer_id</code> points to the customer the legacy key encoded (shop-id + month + year)	all 50 resolve to the correct customer
MT-UNMATCH-1	Unmatched rate after load	count surveys with null <code>repair_customer_id</code> / total loaded surveys	< 2% (SDD Reliability bar); remainder sit in the needs-match queue, none dropped
MT-LEGACY-1	Legacy key retained	assert <code>legacy_match_key</code> preserved on migrated <code>repair_customers</code>	retained for reconciliation only, never used as a live join key post-cutover

5.3 Migration test sources

Tests source from the DDR CSV exports plus the FileMaker Data API read path, never a developer workstation path (SDD gotcha). The worker must tolerate the absence of the known stale external references (BridgeSystem path, EmployeeSatisfaction, Import Flush2, dormant `PSG_SurveyExport_07.1`) without failing the load.

6. PII Review Gates

PII review is a manual sign-off, not an automated test. Each gate blocks its milestone until signed.

Gate	Milestone	Scope of review	Sign-off
PII-1	v1.1	New ops tables (<code>repair_customers</code> , <code>survey_responses</code> extensions, <code>letter_eligibility</code>) follow <code>psg_sensitive_pii_*</code> redaction; RLS clamps customer data to authorized shops; <code>security_profiles.functions_jsonb</code> gates ops functions; survey <code>raw_payload</code> redacted.	Manual sign-off recorded before v1.1 ships
PII-2	v1.3	Production data path (batch documents, mail-merge content, <code>mail_vendor_jobs</code>) carries end-consumer PII to Lob/SendGrid safely; vendor secrets encrypted at rest (pgsodium); no PII in logs.	Manual sign-off recorded before v1.3 cutover
PII-3	v2.0	Final convergence PII audit across customer + ops + internal paths; redaction parity with existing patterns confirmed; <code>access_audit</code> append-only trail verified.	Manual sign-off recorded before public launch posture

Each sign-off uses a written checklist attached to the milestone, names the reviewer, and is the gate of record for the SDD Security requirement and the PLANNING PII review gate.

7. E2E Happy Paths (Playwright)

One ops happy path per milestone, browser-driven against a seeded staging stack, with an LCP assertion on `/ops/*` and `/ops/reports/*` (< 3s).

ID	Milestone	Happy path	Key assertions
E2E-OPS	v1.1	Create company → add employees → import RO	company persists; import validates against <code>import_templates</code> ; RO appears; LCP < 3s
E2E-PROD	v1.3	Company has program → repair customer assigned → batch generated → printed → moved to historical → reprintable	batch builds; print is idempotent on re-run; Lob test-mode webhook updates status; reprint writes audit
E2E-RPT	v1.4	Open <code>/ops/reports</code> → run a survey-response report (e.g. Mis-Fire) over a 12-month range → export	report renders with correct <code>alert_class</code> filter; export downloads; LCP < 3s; second run hits Redis cache
E2E-CONV	v2.0	Convergence suite across customer + ops + internal happy paths	all milestone happy paths pass together; AEGIS + PII final review green

8. Test Data and Environments

- Test Supabase project: a dedicated branch of `gylkkzmcmbdftxieyabw` with the SDD migrations applied. Never run integration tests against production.
- Seed fixtures: roles, modules, `security_profiles`, master data via `node apps/psg-hub/scripts/seed.mjs`, plus a fixture set of companies (varying `referral_tracking_enabled`), repair customers (some on `credit_hold`), surveys spanning all six alert classes, and a batch with mixed print state.
- External services: Lob test mode for the v1.3 E2E path; SendGrid, Twilio, and the FileMaker Data API mocked at unit and integration levels.
- Full-population timing fixtures: a synthetic dataset sized to the DDR counts ($\approx 281\text{K}$ customers, $\approx 334\text{K}$ surveys) to assert cron completion < 5 min.

9. Traceability Summary

SDD Acceptance Criterion	EARS AC	Primary test cases
Resolve FK + set <code>alert_class</code>	AC-S1	UT-CLS-1..6b, IT-INGEST-1/2
Unmatched survey retained + queued	AC-S2	UT-MATCH-1/2, IT-INGEST-3
Exclude open-alert customer + record suppression	AC-E1	UT-SUP-1/2/3, IT-ELIG-1
Never duplicate a letter	AC-E2	UT-ELIG-IDEM-1/2/3, IT-ELIG-2
Print only unprinted + stamp	AC-P1/P2	UT-PRINT-1/2/3, IT-PRINT-1/2
Lob status updates job	AC-P3/P4	IT-LOB-1/2/3, UT-LOB-REPLAY
26 parameter-driven reports	AC-R1/R2	IT-RPT-1/2/3, E2E-RPT
Migration loads + reconciles	AC-M1/M2/M3	MT-COUNT-1..6, MT-LINK-1, MT-UNMATCH-1

Historical Data Migration Runbook (v1.3.5 optional add-on)

This runbook covers the optional historical data migration described in ADR-5 of the Solution Design Document (`solution-design.md`). It is step-by-step and operational. Read the SDD first, especially the Deployment View migration sequencing, ADR-5, and the migration worker directory map.

1. Goal and when this runs

1.1 Goal

1. Load historical FileMaker Advantage records into the psg-hub shared Supabase schema so PSG keeps a single source of historical continuity after FileMaker retires.
2. Resolve the FileMaker composite string match key into real `survey_responses.repair_customer_id` foreign keys, and keep `legacy_match_key` for reconciliation.
3. Reconcile loaded counts against the DDR inventory (repair_customers $\approx$ 281K, survey_responses $\approx$ 334K) before anyone relies on the data.

1.2 When this runs

1. This is v1.3.5, an optional add-on. It runs only if the business needs historical continuity in psg-hub.
2. Functional replacement does not depend on it. The customer track (v0.1–v0.4) and the ops backbone (v1.1 $\rightarrow$ v1.4) ship and operate without this migration.
3. FileMaker stays the authoritative daily driver until the v1.3 cutover. After v1.3 cutover, FileMaker is read-only for history.
4. Run the historical backfill only after the v1.3 functional cutover is complete and stable. See section 8 for the cutover ordering rule.
5. If the business decides historical data is not required in psg-hub, do not run this runbook. FileMaker read-only access satisfies the audit need.

2. Sources and extract method

The migration draws from three distinct sources. Structure comes from the DDR exports; live record data comes from the FileMaker Data API and the vendor `xlsx`.

2.1 Schema and field mapping (DDR, structure only)

1. The DDR CSV exports live at `agents/CI0/Filemaker/2025-05-28/Exports for Reference/` (Scripts, Steps, Fields, Tables, Relationships, Layouts CSVs, plus the raw `_fmp12.xml` DDRs).
2. Use these for table structure, field names, indexed fields, value lists, and the match-key calculation definitions. They are the migration source for structure, not for record data.
3. Confirm field-to-column mapping against the DDR before each entity load. The `fm_transform/` mapping tables are built from these CSVs.

2.2 Live records (FileMaker Data API)

1. Live repair customers, repair orders, estimates, and production history extract through the FileMaker Data API on the `psgweb.me` host.
2. Configure `FILEMAKER_DATA_API_*` env vars (host, database, account, password). The Data API session token is short-lived; the extract layer re-authenticates and pages through layouts.

3. The Data API is read-only for migration. No write-back to FileMaker at any point (see section 8.4).

2.3 Survey and repair responses (vendor xlsx)

1. The vendor xlsx exports for Advantage repair and survey responses live in `psg-data-lake/Export/` (for example `advantage-survey-responses*.xlsx`, the same source v0.3 used to promote `survey_responses`).
2. Use the xlsx as the survey/repair response source where the Data API does not expose a clean per-survey row, and to cross-check Data API counts.
3. Where the xlsx and the Data API disagree on a survey row, treat the Data API as the record-level source of truth and log the variance.

2.4 Worker location

1. The migration worker lives in `psg-data-lake/`: `fm_migrate.py` (orchestrator), `fm_extract/` (DDR CSV + Data API + xlsx readers), and `fm_transform/` (match-key resolution, PII redaction, dedupe).
2. Run via the SDD project command:

```
python psg-data-lake/fm_migrate.py --entity repair_customers # one entity at a time
```

1. The worker runs in the Vercel Sandbox or the `psg-data-lake` venv. Target is Supabase project `gylkkzmcmbdftxieyabw`.

3. Entity load order

Load entities in dependency order. Each step precedes the next because the later entity carries a foreign key into the earlier one. Loading out of order produces orphan foreign keys and forces re-runs. Run one entity at a time and reconcile its count (section 7) before starting the next.

3.1 Master data first

1. `vehicles` — referenced by `repair_orders.vehicle_id`. No inbound dependency, load first.
2. `insurance_companies` — referenced by `repair_orders.insurance_company_id` and by `insurance_agents`.
3. `insurance_agents` — references `insurance_companies` (via `insurance_company_ids[]`); referenced by `repair_orders.insurance_agent_id`. Load after `insurance_companies`.
4. `products` — referenced by `company_programs.product_id` and `production_documents.product_id`.
5. `items` — referenced by `products.items_jsonb` content. Load alongside `products`.

Rationale: master data has no dependency on transactional records, but every transactional record points back to it. Master data must exist before any row that references it, or the foreign key resolves to null.

3.2 Companies

1. Load `companies` (the system-generated Shop ID is the primary internal ops entity).
2. Rationale: `employees`, `repair_customers`, `repair_orders`, `estimates`, and `company_programs` all carry `company_id`. Companies must exist first.

3.3 Employees

1. Load `employees` (references `company_id`).

2. Rationale: companies must exist. Employees are referenced by report attribution (estimator, body tech, painter performance reports) but carry no inbound FK from the core transactional load, so they slot in after companies and before transactional records.

3.4 Repair customers

1. Load `repair_customers` (references `company_id`).
2. Set `legacy_match_key` from the FileMaker `RC_MatchField_Master` value during this load. This is required for survey match-key resolution in section 4 and for reconciliation in section 7.
3. Seed `referral_tracking_enabled` and `credit_hold` here (see section 6.4); both are read before survey classification.
4. Rationale: `repair_orders`, `estimates`, and `survey_responses` all carry `repair_customer_id`. Repair customers must exist before any of them, and the match key must be present before surveys are linked.

3.5 Repair orders

1. Load `repair_orders` (references `repair_customer_id`, `company_id`, `vehicle_id`, `insurance_company_id`, `insurance_agent_id`).
2. Rationale: repair orders depend on repair customers and all of master data. They also carry the RO completion date used by eligibility, so they precede any eligibility-related history.

3.6 Estimates

1. Load `estimates` (references `repair_customer_id`, `company_id`).
2. Rationale: estimates depend on repair customers and companies. They have no inbound FK from surveys or production, so they load after ROs and before surveys.

3.7 Survey responses

1. Load `survey_responses` (references `repair_customer_id`, nullable until matched, plus `shop_id`).
2. Resolve `repair_customer_id` from the legacy match key during this load (section 4). Leave the FK null and queue the row when no match resolves; never drop it.
3. Classify `alert_class` only after `referral_tracking_enabled` and `credit_hold` are seeded (section 6.4).
4. Rationale: surveys point at repair customers. Repair customers and their `legacy_match_key` values must already exist for the FK to resolve.

3.8 Production history

1. Load production history (`production_batches`, `production_documents`, `production_reprint_log`, and any `mail_vendor_jobs` history) last.
2. Rationale: `production_documents` references `company_id`, `product_id`, and `repair_customer_id`. Every one of those must already be loaded. Production history is the leaf of the dependency graph, so it loads last.

4. Match-key resolution

The FileMaker composite string match key becomes a real foreign key (ADR-2). This is the load-bearing transform of the survey load.

4.1 What the legacy key is

1. The FileMaker key is a composite string built from shop/master id + record creation month + creation year.
2. On the Advantage side: `RC_MatchField_Master` is the raw shop/master key; `RC_MatchField_survey = RC_MatchField_Master & RC_CreationDate_Month & RC_CreationDate_Year`.
3. On the Survey side: `S_RC_ShopID` is the shop key; `S_RC_MatchField_RepairCust = S_RC_ShopID & S_CreationDate_Month & S_CreationDate_Year`.
4. The live join in FileMaker is `Repair_Customer::RC_MatchField_survey = Survey Input::S_RC_MatchField_RepairCust`.

4.2 How resolution works

1. During the repair_customers load (section 3.4), store `repair_customers.legacy_match_key` as the composite survey-form key (shop/master id + creation month + creation year), matching the Advantage `RC_MatchField_survey` shape.
2. During the survey load, compute the same composite key on the survey side (`S_RC_ShopID` + creation month + creation year).
3. Resolve `survey_responses.repair_customer_id` by joining the computed survey key to `repair_customers.legacy_match_key`.
4. Keep `legacy_match_key` on `repair_customers` after migration. It exists for reconciliation only, not for runtime joins.

4.3 Build the needs-match queue

1. When a survey key resolves to exactly one repair customer, set `repair_customer_id` and proceed.
2. When a survey key resolves to zero repair customers, leave `repair_customer_id` null and place the row in the needs-match queue (the ops "needs match" surface). Never drop the survey.
3. When a survey key resolves to more than one repair customer (ambiguous month/year/shop collision), leave `repair_customer_id` null, queue the row, and tag it ambiguous so an account manager picks the correct link.
4. The needs-match queue is the explicit, queryable replacement for the FileMaker failure mode where a shop-id or month/year mismatch silently broke the survey-to-customer link.
5. Target unmatched-survey rate below 2 percent after reconciliation (SDD reliability bar). Above 2 percent, stop and investigate the key construction before continuing.

5. Transform rules

All transforms live in `fm_transform/`. Apply them in this order during each entity load.

5.1 PII redaction

1. Repair customers and survey content are end-consumer PII (names, addresses, phones, emails, survey free text).
2. Apply the existing `psg_sensitive_pii_*` redaction patterns to every loaded field that those patterns cover. Do not invent new redaction logic.
3. Redact `survey_responses.raw_payload` before write, matching the v0.3 promotion behavior.
4. Keep secrets encrypted at rest (pgsodium) where the schema already does so. The migration writes no plaintext secret.

5. Manual PII sign-off is required at the migration milestone, consistent with the v1.1 / v1.3 / v2.0 PII review gates.

5.2 Dedupe

1. The FileMaker files carry accumulated redundant definitions and duplicate external data source entries. Expect duplicate rows in the extract.
2. Dedupe master data on natural keys (vehicle make+model; insurance company name; product name) before load so transactional foreign keys resolve to one canonical row.
3. Dedupe repair_customers on `legacy_match_key` plus identity fields. Collapse exact duplicates; queue near-duplicates for manual review rather than guessing.
4. Dedupe survey_responses on the source survey row id plus computed match key so a re-run does not double-insert.

5.3 Seed referral_tracking_enabled and credit_hold before classification

1. The `referral` vs `misfire` branch depends on per-company `referral_tracking_enabled` and the customer `credit_hold` flag. In FileMaker these were read cross-file from `Master Client::M_ReferralNoted_flag` and `M_CreditHold_flag`.
2. Seed `company_programs.referral_tracking_enabled` (and `repair_customers.referral_tracking_enabled` where the per-customer copy applies) from the FileMaker `M_ReferralNoted_flag` value during the companies / repair_customers load.
3. Seed `repair_customers.credit_hold` from the FileMaker `M_CreditHold_flag` value during the repair_customers load.
4. These seeds must complete before survey classification runs in section 3.7. If they are missing, every referral-eligible survey misclassifies as misfire.

5.4 Survey classification (after seeds)

1. Classify each loaded survey into `alert_class` (perfect / misfire / hotspot / unresolved / referral / none) using the SDD classification rules, which encode the DDR v3 LIVE logic.
2. Confirm the v3 LIVE alert logic is the authoritative generation before running classification. FileMaker carried three live generations (v2, WIP, v3 LIVE); only v3 LIVE is wired to the Web triggers.
3. Map the FileMaker idempotency stamps to columns: `SQ_Alert_PostDate_*` to `alert_posted_at`, and `S_RC_Referral_Letter_Post` to `referral_letter_posted_at`. Historical surveys that were already posted in FileMaker arrive with these timestamps set, so psg-hub does not re-send for already-handled history.

6. Reconciliation

Reconcile every entity before starting the next. Treat reconciliation as a gate, not a report.

6.1 Count targets

1. `repair_customers` $\approx$ 281,077 (DDR Repair Customer table inventory, 5/27/2025).
2. `survey_responses` $\approx$ 334,269 (DDR Survey Input table inventory, 5/27/2025).
3. For master data, companies, employees, repair_orders, estimates, and production history, reconcile loaded counts against the DDR table inventory and the source extract counts for that entity.

6.2 Variance tolerance

1. Counts are dated 5/27/2025 and the live FileMaker has changed since. Expect small drift. Treat a per-entity variance within plus or minus 2 percent of the DDR target as within tolerance.
2. Unmatched-survey rate must be below 2 percent (section 4.3).
3. Record the actual loaded count, the DDR target, and the variance percentage for each entity in the run log.

6.3 50-link customer-to-survey spot check

1. Sample 50 `survey_responses` rows that resolved a `repair_customer_id`.
2. For each, confirm the resolved repair customer matches the source FileMaker survey-to-repair link (shop id, creation month, creation year all agree).
3. All 50 must resolve correctly. Any incorrect link means the match-key construction is wrong; stop and fix the key before trusting the survey load.

6.4 What to do on mismatch

1. Within tolerance: record the variance, mark the entity reconciled, proceed to the next entity.
2. Outside tolerance (count drift beyond plus or minus 2 percent): stop. Diff the loaded set against the source extract to find dropped or duplicated rows. Re-run the entity load (it is idempotent, section 9) after fixing the cause. Do not start the next entity.
3. Unmatched-survey rate at or above 2 percent: stop. Inspect the needs-match queue, verify `legacy_match_key` was populated on `repair_customers`, verify the survey-side key construction, fix, and re-resolve.
4. Spot-check failure: stop. The FK resolution is unsafe. Fix match-key logic and re-run the survey load.

7. Validation SQL examples

Run these against Supabase project `gylkzmcmbdftxieyabw` after each entity load. They confirm counts, catch orphan foreign keys, and catch duplicates.

7.1 Count per table

```
SELECT 'repair_customers' AS entity, count(*) AS rows FROM repair_customers
UNION ALL SELECT 'repair_orders', count(*) FROM repair_orders
UNION ALL SELECT 'estimates', count(*) FROM estimates
UNION ALL SELECT 'survey_responses', count(*) FROM survey_responses
UNION ALL SELECT 'companies', count(*) FROM companies
UNION ALL SELECT 'employees', count(*) FROM employees
UNION ALL SELECT 'production_documents', count(*) FROM production_documents
ORDER BY entity;
```

7.2 Orphan foreign key checks

```
-- Survey rows that have a repair_customer_id pointing at a missing customer.
SELECT count(*) AS orphan_survey_fk
FROM survey_responses sr
LEFT JOIN repair_customers rc ON rc.id = sr.repair_customer_id
WHERE sr.repair_customer_id IS NOT NULL
      AND rc.id IS NULL;

-- Repair orders pointing at missing parents (customer, company, or master data).
SELECT count(*) AS orphan_ro_customer
```

```

FROM repair_orders ro
LEFT JOIN repair_customers rc ON rc.id = ro.repair_customer_id
WHERE rc.id IS NULL;

SELECT count(*) AS orphan_ro_company
FROM repair_orders ro
LEFT JOIN companies c ON c.id = ro.company_id
WHERE c.id IS NULL;

SELECT count(*) AS orphan_ro_vehicle
FROM repair_orders ro
LEFT JOIN vehicles v ON v.id = ro.vehicle_id
WHERE ro.vehicle_id IS NOT NULL AND v.id IS NULL;

-- Production documents pointing at missing parents.
SELECT count(*) AS orphan_doc_customer
FROM production_documents pd
LEFT JOIN repair_customers rc ON rc.id = pd.repair_customer_id
WHERE pd.repair_customer_id IS NOT NULL AND rc.id IS NULL;

```

Every orphan count must be zero. A non-zero orphan count means an entity loaded out of order (section 3); re-run after loading the missing parent.

7.3 Needs-match queue size and unmatched rate

```

-- Unmatched survey count and rate.
SELECT
  count(*) FILTER (WHERE repair_customer_id IS NULL) AS unmatched,
  count(*) AS total,
  round(100.0 * count(*) FILTER (WHERE repair_customer_id IS NULL) / nullif(count(*), 0), 2) AS
  unmatched_pct
FROM survey_responses;

```

`unmatched_pct` must be below 2.0.

7.4 Duplicate checks

```

-- Duplicate repair customers on the legacy match key (should be 0 rows after dedupe).
SELECT legacy_match_key, count(*) AS dupes
FROM repair_customers
WHERE legacy_match_key IS NOT NULL
GROUP BY legacy_match_key
HAVING count(*) > 1
ORDER BY dupes DESC
LIMIT 50;

-- Duplicate production documents on print_id (print_id is unique by design).
SELECT print_id, count(*) AS dupes
FROM production_documents
GROUP BY print_id
HAVING count(*) > 1;

-- Duplicate master data (example: vehicles on make+model).
SELECT make, model, count(*) AS dupes
FROM vehicles
GROUP BY make, model
HAVING count(*) > 1
ORDER BY dupes DESC;

```

7.5 Seed verification before classification

```
-- Confirm referral/credit-hold seeds landed before survey classification runs.
SELECT
  count(*) FILTER (WHERE referral_tracking_enabled IS NULL) AS programs_missing_referral_flag
FROM company_programs;

SELECT
  count(*) FILTER (WHERE credit_hold IS NULL) AS customers_missing_credit_hold
FROM repair_customers;
```

Both must be zero before running survey classification (section 5.3).

8. Parallel run and cutover

8.1 Parallel run authority

1. During parallel run (v1.1 → v1.3), FileMaker is authoritative. psg-hub is downstream only.
2. psg-hub writes its own data during this period. It does not write back to FileMaker.

8.2 After cutover

1. At v1.3 cutover, point the PSG production team to psg-hub for all new production.
2. FileMaker becomes read-only for historical access. It is no longer a daily driver.

8.3 Order of functional cutover vs historical backfill

1. Do the functional cutover (a) first. Complete and stabilize the v1.3 Production cutover so psg-hub is the daily driver for new work.
2. Do the historical backfill (b) second. Run this v1.3.5 migration after the functional cutover is stable.
3. Rationale: functional replacement does not depend on historical data (ADR-5). Cutting over first means the daily driver is live and verified before the large historical load runs, and the backfill cannot block or destabilize the cutover.
4. During the gap between (a) and (b), current data is in psg-hub and deep history is in read-only FileMaker. This is expected and acceptable.

8.4 No write-back

1. There is no two-way sync. psg-hub never writes into FileMaker.
2. The FileMaker Data API is used read-only, for extract only, during both parallel run and this migration.

9. Rollback and idempotency

9.1 Idempotency

1. Every entity load is re-runnable. Loads are keyed on legacy ids (`legacy_match_key` for `repair_customers`; source row ids for surveys, ROs, estimates; `print_id` for production documents; natural keys for master data).
2. Loads upsert on the legacy key. Re-running an entity updates existing rows in place and does not create duplicates.
3. Survey classification and FK resolution re-run safely: re-resolution overwrites the same `repair_customer_id` and `alert_class`, and the post-date stamps prevent re-sending already-handled history.

9.2 How to abort safely

1. Abort between entities, not mid-entity, whenever possible. The load order (section 3) gives clean stopping points.
2. To abort a single entity load mid-run, stop the worker. Because the load is an idempotent upsert keyed on legacy ids, partial progress is consistent. Re-run the same entity from the start to finish it.
3. Do not run the next entity after an aborted entity until the aborted entity reconciles (section 6).
4. Full unwind of a v1.3.5 migration: because the schema additions are additive and FileMaker remains read-only and authoritative for history, a UI-level rollback leaves loaded data intact. If the loaded data itself must be removed, delete by the entity's legacy-id set in reverse load order (production history → surveys → estimates → ROs → repair_customers → employees → companies → master data) so no foreign key is orphaned during teardown.
5. Migrations are forward-only at the schema level (per the SDD breaking-change policy). Rollback means reverting application behavior and, if required, deleting loaded rows, not reversing schema migrations.

10. Operational notes

10.1 Stale FileMaker external references to ignore

These references exist in the FileMaker files but are stale, dormant, or broken. Do not recreate them and do not let their absence fail the extract.

1. `BridgeSystem` — hardcoded to a developer desktop path (`../Users/stevesch/Desktop/BridgeSystem`). Ignore.
2. `EmployeeSatisfaction` — external file not in the export set. Ignore.
3. `Import Flush2` — stale ETL reference. Ignore. The 325 per-shop importers are replaced by `import_templates` , not migrated.
4. `PSG_SurveyExport_07.1` — declared in both Advantage and Survey but no script, import, or export uses it. Dormant. Ignore.
5. `Phoenix Solutions Group` — flagged Unknown, empty path, a dead reference. Ignore.

The migration extract must tolerate the absence of all of these. A "file not found" prompt from any of them is expected, not a failure.

10.2 No machine-specific import paths

1. Do not source any data from machine-specific local paths (for example `/Users/ryan/Downloads/1.fmp12`). Those are fragile workstation artifacts.
2. Source structure from the DDR exports at `agents/CIO/Filemaker/2025-05-28/Exports for Reference/` , live records from the FileMaker Data API, and survey/repair responses from `psg-data-lake/Export/` . Nothing else.

10.3 Run discipline

1. Run one entity at a time (`--entity <name>`), in the order of section 3.
2. Reconcile (section 6) and run validation SQL (section 7) after each entity, before the next.
3. Record loaded count, DDR target, variance, and unmatched rate per entity in the run log.
4. Stop on any out-of-tolerance variance, non-zero orphan count, or spot-check failure. Fix the cause, re-run the idempotent load, then continue.

ADR-001: Re-platform FileMaker behavior, do not port code or schema 1:1

Status

Proposed

Context

PSG runs the FileMaker Advantage Program (four files: Advantage, Survey, Import Flush, Web) as its operational daily driver. The DDR analysis of those files exposed heavy structural duplication: 325 per-shop import scripts, triplicate per-letter scripts (Preview/Print/Loop), and three live generations of survey alert scripts (v2/WIP/v3 LIVE). It also confirmed there is almost no event layer to preserve (0 layout triggers, 12 `OnObjectExit` validations, 1 in-file PSOS). Meanwhile `PLANNING.md` already commits to replacing FileMaker and already drafts the target tables (`repair_customers` , `survey_responses` , `production_batches` , the 26 reports) and the cron surfaces. The one gap is the behavioral spec for what those modules must do.

A direct FileMaker-to-SQL lift-and-shift would reproduce that duplication and import a schema shaped by FileMaker constraints rather than by the psg-hub domain model.

Decision

Re-platform behavior, do not port code or schema 1:1. Rebuild each FileMaker subsystem as a psg-hub module (Ops, Surveys, Production, Reports) on the already-planned Supabase schema. Use the DDR analysis as the behavioral source of truth for the rules that must be preserved exactly: letter eligibility windows, the Perfect/Misfire/HotSpot/Unresolved/Referral alert classifications and thresholds, the suppression rule, and the printed/posted idempotency semantics. Collapse the duplicated FileMaker constructs into single, mapping-driven or status-driven implementations.

Consequences

Positive

- One importer (`import_templates`) replaces 325 per-shop scripts.
- One alert classifier replaces three live generations of alert scripts.
- Batch + template + status replaces the triplicate per-letter scripts.
- The schema is shaped by the psg-hub domain model, not by FileMaker layout constraints, and integrates cleanly with the shared Supabase project.

Negative / trade-offs

- More upfront design work than a mechanical lift-and-shift.
- Correctness depends on the DDR being an accurate behavioral record; the v3 LIVE alert logic must be confirmed authoritative before it is encoded.
- A behavioral parity check is required at cutover to prove the rebuild matches FileMaker output.

Alternatives considered

- **Lift-and-shift FileMaker schema and scripts into Postgres.** Rejected: it reproduces the duplication the DDR flagged as technical debt and locks psg-hub into a FileMaker-shaped data model.
- **Keep FileMaker and bolt psg-hub on as a read-only mirror.** Rejected: it does not retire the legacy system, which is the explicit goal of the milestone plan.

ADR-002: Replace the composite string match key with a real foreign key

Status

Proposed

Context

In FileMaker, a survey response is linked to its repair customer through a composite string match key built from shop id + month + year (`RC_MatchField_Master`). The DDR cross-file analysis showed this link breaks silently: any mismatch in shop id, month, or year produces no error, just an unmatched survey that disappears from the feedback loop. Because the survey↔letter loop drives customer contact and revenue (suppression and referral letters depend on it), a silent break is a correctness and revenue risk, not just a data-quality nuisance.

psg-hub stores survey responses and repair customers as first-class rows in one Supabase schema, so a referential link is available.

Decision

Replace the composite string match key with a real foreign key: `survey_responses.repair_customer_id` references `repair_customers(id)`. The FK is nullable until matched. A survey that cannot be resolved is retained with a null FK and surfaced in a needs-match ops queue; it is never silently dropped. The legacy `RC_MatchField_Master` string is kept as `repair_customers.legacy_match_key` (indexed) for migration reconciliation only, and as a fallback resolver during the v1.3.5 historical load.

Consequences

Positive

- Broken links become visible and queryable instead of silent.
- The needs-match queue gives ops a place to resolve unmatched surveys, with a target unmatched rate under 2% after migration reconciliation.
- Referential integrity is enforced by the database, not by string assembly in application logic.

Negative / trade-offs

- The v1.3.5 migration must resolve every legacy composite key to a real FK, using `legacy_match_key` as the bridge.
- A nullable FK plus a needs-match workflow is one more state to operate and monitor.

Alternatives considered

- **Keep the composite string key in Postgres.** Rejected: it carries the silent break forward and gives the database no way to enforce the relationship.
- **Hard FK with NOT NULL, reject unmatched surveys at write time.** Rejected: it would drop or block legitimate surveys that arrive before their repair customer is matched, recreating the very data loss this decision removes. Nullable FK plus a needs-match queue preserves every survey.

ADR-003: Move automation to Vercel cron + idempotent Postgres workers

Status

Proposed

Context

All automation in FileMaker Advantage today runs from the Web file: it fires Perform Script on Server (PSOS) on a FileMaker Server schedule, and it tracks incremental work with date-stamp fields (process a record, stamp it, skip already-stamped records on the next run). The DDR confirmed there is no event-trigger layer to preserve (0 layout triggers, 1 in-file PSOS). This makes automation a single point of failure: if the scheduled PSOS does not run, nothing self-heals, and the date-stamp pattern depends on every write succeeding exactly once.

psg-hub targets Vercel hosting with Vercel Cron and a shared Supabase Postgres, and `PLANNING.md` already declares the cron surface (`survey-ingest` , `production-batch-status` , `mail-vendor-poll` , and others).

Decision

Move all automation to Vercel cron plus idempotent Postgres workers. Retire the Web PSOS scheduler and the date-stamp incremental pattern. Each scheduled job is window-based and idempotent: it selects rows in a bounded date window, resolves foreign keys, computes or classifies, and upserts under a unique constraint (`letter_eligibility` unique on customer + letter_kind + period_key; status columns plus `printed_at` guards for production). Idempotency replaces date stamps; a unique constraint plus `on conflict do nothing / do update` prevents double-effects.

Consequences

Positive

- A missed cron self-heals on the next run because each job is window-based, not pointer-based.
- Re-triggering a job cannot double-send: unique constraints and `printed_at` guards make every run idempotent.
- Cron runs emit counts (computed, eligible, suppressed, printed) for observability, which FileMaker lacked.

Negative / trade-offs

- A missed cron only self-heals if the window is wide enough to recover the gap; windows must be sized deliberately.
- Loss of self-healing must still be detected: a freshness alert is required when `computed_at` is stale beyond about 36 hours.
- Service-role cron writes bypass RLS, so the worker code carries the authorization responsibility the database would otherwise enforce.

Alternatives considered

- **Database triggers / pg_cron inside Postgres.** Rejected: it spreads automation logic across the database and the app, and is harder to observe and deploy than route handlers the team already owns.
- **Port the FileMaker date-stamp pattern to Postgres columns.** Rejected: it keeps the brittle exactly-once-write assumption and does not give the self-healing that window-based idempotent runs provide.

ADR-004: Make the survey-alert suppression rule explicit

Status

Proposed

Context

A core rule of the FileMaker feedback loop is suppression: when a customer has an open survey alert, they are removed from routine mail so PSG does not send a cheerful thank-you or warranty letter to an unhappy customer. In FileMaker this was an emergent side effect of an `IsEmpty(survey alert)` guard inside the eligibility finds. The DDR proved the behavior exists but showed it was invisible: nothing in the data recorded that a customer was suppressed, or why. That makes it impossible to audit, easy to break accidentally during a rebuild, and impossible to report on.

Decision

Make suppression explicit and queryable. The `letter_eligibility` table carries a `suppressed_by_alert` boolean alongside `eligible`. When eligibility is computed, a customer with an open survey alert in the window is marked `eligible = false` and `suppressed_by_alert = true`, recording both the outcome and the reason in the row. Suppression becomes a stored, auditable fact instead of an emergent join behavior.

Consequences

Positive

- Suppression is auditable: a reviewer can see exactly who was held back from mail and why.
- It is reportable: the survey-alert and eligibility reports can count suppressed customers directly.
- It is harder to break silently during the rebuild, because the rule is a named column with an acceptance criterion, not a hidden side effect.

Negative / trade-offs

- One extra column and one extra compute step in the eligibility cron.
- The suppression window definition (which alert classes, over what interval) must be specified precisely, because it is now visible and will be checked.

Alternatives considered

- **Keep suppression as an emergent join filter.** Rejected: it reproduces the invisible behavior the DDR flagged and leaves no audit trail.
- **Record suppression only in cron logs, not in the row.** Rejected: logs are not queryable alongside the eligibility data and do not survive as the durable record an audit or report needs.

ADR-005: Historical data migration is optional, via a psg-data-lake worker (v1.3.5)

Status

Proposed

Context

psg-hub replaces FileMaker Advantage as the daily driver, but the functional replacement (v1.1 ops foundation through v1.4 reports) does not inherently require the historical data to move with it. FileMaker holds large history (the DDR counts roughly 281K repair customers and 334K survey responses). Moving that data is a distinct effort with its own risks: match-key resolution to real foreign keys (ADR-2), PII redaction, dedupe, and count reconciliation. `PLANNING.md` already treats this as add-on scope (Q2/Q15 deferred), and the SDD keeps functional replacement off the critical path of data movement.

There is an existing Python ETL home (`psg-data-lake`) and existing extract sources (the DDR CSV exports plus the FileMaker Data API on the psgweb.me host).

Decision

Treat historical data migration as optional v1.3.5 scope, delivered by a `psg-data-lake` Python worker (`fm_migrate.py` : extract → transform → load). It sources from the DDR CSV exports and the FileMaker Data API, resolves legacy match keys to real foreign keys, applies PII redaction, dedupes, and reconciles loaded counts against the DDR before each sequencing step proceeds. Functional replacement (v1.1 through v1.4) ships independently of this worker; v1.3.5 runs only if the business requires historical continuity in psg-hub.

Consequences

Positive

- Cutover is not blocked by data movement; FileMaker can go read-only for history while psg-hub owns current operations.
- The migration reuses the existing `psg-data-lake` runtime and the DDR exports, rather than standing up new infrastructure.
- Loaded counts reconcile to the DDR (repair_customers ~281K, survey_responses ~334K) with a 50-link spot check, giving a measurable integrity gate.

Negative / trade-offs

- A period exists where current data lives in psg-hub and deep history lives in read-only FileMaker, so cross-period reports must account for two sources until the migration completes.
- If the business later requires history, the migration becomes a real project with its own timeline; deferring it now defers that cost, it does not remove it.

Alternatives considered

- **Migrate all history before cutover (block cutover on data movement).** Rejected: it couples functional replacement to a large, risky ETL and delays retiring FileMaker as the daily driver.

- **Never migrate; leave all history in read-only FileMaker permanently.** Rejected as the default: it may be acceptable for some history, but hard-coding it forecloses historical continuity the business may need. Making migration an optional, ready-to-run phase keeps the choice open.